Perceptrony wielowarstwowe

Martyna Sadowska, nr indeksu 333157

Wstęp

W tym projekcie wprowadzona została implementacja perceptronów wielowarstwowych wraz z różnymi modyfikacjami. W kolejnych etapach była ona stopniowo rozbudowywana o kolejne elementy. Testowany był wpływ zmian wartości parametrów sieci oraz jej działanie na wielu zbiorach danych dotyczących zarówno problemu regresji, jak i klasyfikacji. Implementacja algorytmów od podstaw pozwoliła na dogłębną analizę wewnętrznych mechanizmów uczenia, takich jak algorytmy optymalizacji, różnorodne funkcje aktywacji czy techniki zapobiegające przeuczeniu.

Bazowa implementacja

W pierwszym etapie stworzona została bazowa implementacja sieci neuronowej typu MLP. Zaimplementowane zostały bazowe klasy warstwy liniowej oraz funkcji sigmoid, których obiekty stanowią część głównej klasy MLP - modelu sieci. Wywołanie funkcji forward na modelu powoduje wywołanie forward na każdej warstwie i funkcji aktywacji. Przy deklaracji każdej warstwy liniowej należy podać liczbę neuronów wejściowych i wyjściowych. Automatycznie ustawiane są wagi 1 i bias 0 dla każdego elementu, ale można je zmienić po deklaracji obiektu.

Cała analiza na tym etapie była wykonywana dla dwóch zbiorów danych: square-simple i steps-large oraz dla trzech architektur sieci: z jedną warstwą ukrytą z 5 neuronami, z jedną warstwą ukrytą z 10 neuronami i z dwoma warstwami ukrytymi po 5 neuronów.

Na tym etapie pracy odpowiednie parametry sieci były szukane przy wykorzystaniu biblioteki PyTorch. Dla każdego zbioru i każdej z architektur dane były normalizowane i trenowane pod najmniejszą wartość MSE. Długość treningu dla zbioru square-simple wyniosła 80 000 kroków, a dla steps-large 60 000. Dla tych danych wyniki wychodziły wystarczająco zbliżone przy niedużym czasie wykonania. Współczynnik uczenia został ustawiony na 0.01, co przyspieszyło dojście do lepszych wyników.

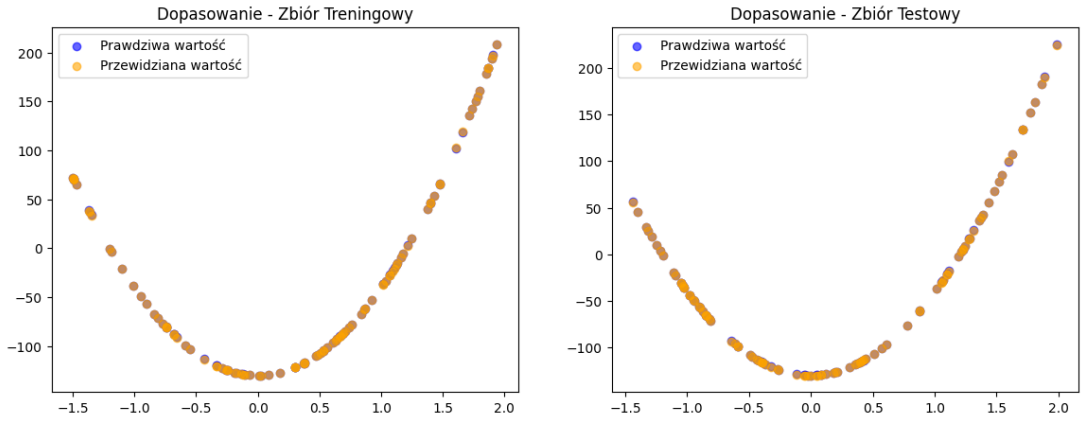
Z modeli wytrenowanych w PyTorchu zostały wyciągnięte wagi i biasy, które zostały użyte w tworzonej przez nas sieci. Po treningu dawała ona takie same wyniki jak PyTorch, co weryfikuje poprawność jej działania.

W tabeli 1 umieszczone są przybliżone nieznormalizowane finalne wartości MSE otrzymane po trenowaniu sieci dla konkretnych zbiorów danych. Można zauważyć, że dla zbioru square-simple najlepsze wyniki daje model z 1 warstwą z 5 neuronami, a dla zbioru steps-large model z 2 warstwami po 5 neuronów. Zauważalna jest różnica między wartościami MSE dla dwóch zbiorów - sieć dużo lepiej radzi sobie na zbiorze square-simple.

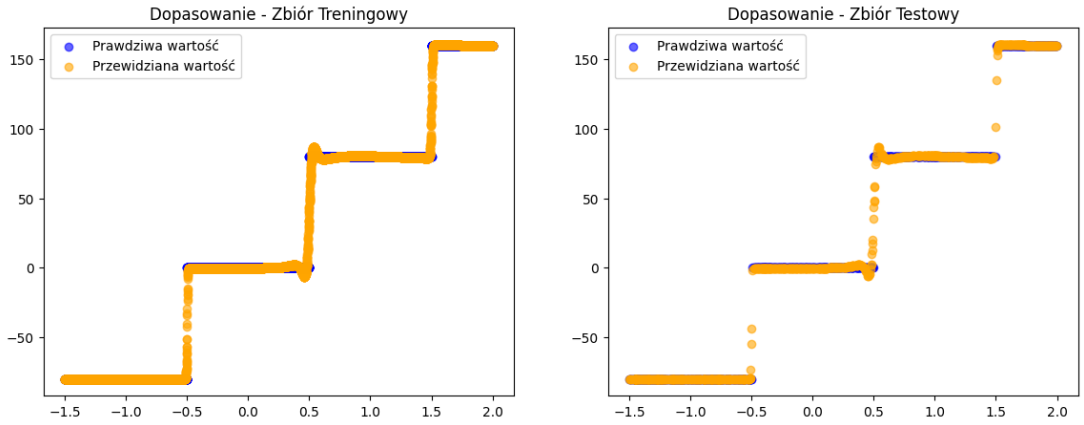
Zbiór danych	Architektura	MSE na zbiorze treningowym	MSE na zbiorze testowym
square-simple	1 - 5 - 1	0.0293	0.0344
	1 - 10 - 1	0.0273	0.0360
	1 - 5 - 5 - 1	0.640	0.633
steps-large	1 - 5 - 1	119.975	115.761
	1 - 10 - 1	120.971	120.074
	1 - 5 - 5 - 1	11.001	10.327

Tabela 1: Przybliżone nieznormalizowane finalne wartości funkcji straty dla konkretnych modeli i zbiorów danych.

Na wykresach zaprezentowane jest działanie sieci dla wybranych modeli dla dwóch zbiorów - tych, które dawały najlepsze wyniki. Przedstawione są na nich rzeczywiste wartości danych oraz wartości przewidziane przez sieć. Na wykresie 1 widoczne są dane przewidziane przez sieć z 1 warstwą z 5 neuronami dla danych square-simple, a na wykresie 2 widoczne są dane przewidziane przez sieć z 2 warstwami po 5 neuronów dla danych steps-large.



Rysunek 1: Wartości rzeczywiste i przewidziane przez sieć z 1 warstwą z 5 neuronami dla danych square-simple.



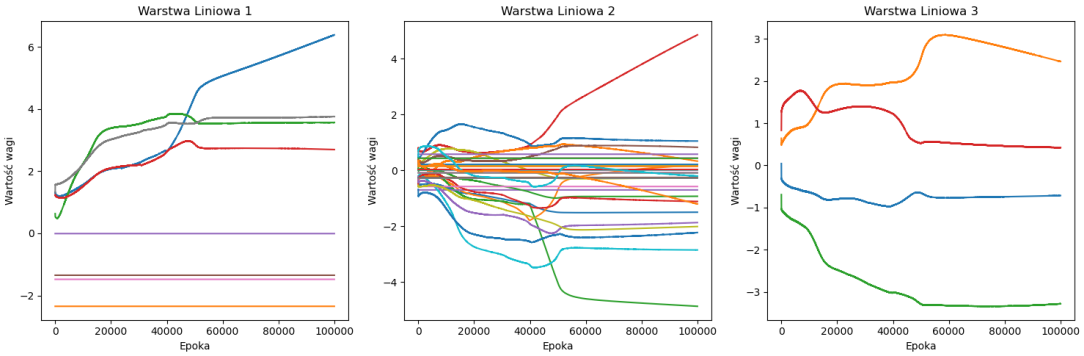
Rysunek 2: Wartości rzeczywiste i przewidziane przez sieć z 2 warstwami po 5 neuronów dla danych steps-large.

Implementacja propagacji wstecznej błędu

Na tym etapie zaimplementowana została propagacja wsteczna błędu do ustalenia wag modelu. Początkową inicjalizację wag zmieniono na wybór metody Xaviera, He lub losowania z rozkładu jednostajnego na [0,1] oraz dodano metodę backward do klasy warstwy liniowej i funkcji aktywacji.

Metoda ucząca model i aktualizująca wagi może działać na dwa sposoby - aktualizować wagi po prezentacji kolejnych porcji (metoda Mini-Batch) lub po prezentacji wszystkich wzorców (wtedy rozmiar batcha to rozmiar danych wejściowych) (metoda Full-Batch).

Na tym etapie analizowane były 3 zbiory: square-simple, steps-small i multimodal-large. Architektura sieci, szybkość uczenia i wielkości porcji dla każdego z nich zostały wybrane poprzez testy wielu różnych możliwości, analizę problemów oraz wykresów zmian wag w czasie. Wykres 3 jest przykładowym wykresem wag dla warstw modelu wywołanego na zbiorze steps-small. Widać na nim, że część z wag w pierwszej warstwie nie zmieniała się wraz z kolejnymi epokami, natomiast reszta stopniowo poprawiała swoją wartość.



Rysunek 3: Zmiana wartości wag w kolejnych epokach w modelu dla danych steps-small używającego metody Mini-Batch.

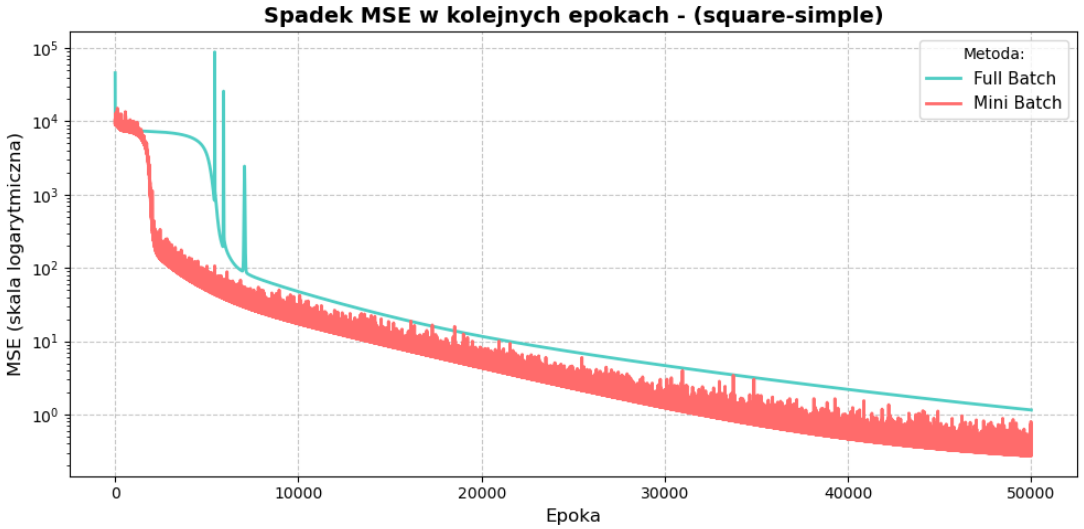
Wartości ostatecznie użyte w modelach dla poszczególnych zbiorów danych umieszczone są w tabeli 2. W tabeli 3 są umieszczone przybliżone nieznormalizowane finalne wartości MSE otrzymane po trenowaniu sieci dla konkretnych zbiorów danych metodą Mini-Batch oraz Full-Batch. Na wykresie 4 porównana jest zbieżność modeli korzystających z metody Mini-Batch i Full-Batch na danych square-simple. Widać na nim, że metoda Full-Batch cały czas szybciej zbiegała do niskich wartości MSE. Podobną zależność można zauważyć na wykresach dla zbioru steps-small (5) oraz multimodal-large (6).

Zbiór danych	Architektura	Funkcje aktywacji	Inicjalizacje wag	Szybkość uczenia	Liczba epok	Wielkość porcji
square-simple	1 - 16 - 8 - 1	Sigmoid, Sigmoid	Xavier, Xavier, Xavier	0.5	50 000	całe dane
				0.2	50 000	16
steps-small	1 - 8 - 4 - 1	ReLu, Sigmoid	He, Xavier, Xavier	0.6	1 000 000	całe dane
				0.1	100 000	2
multimodal-large	1 - 4 - 8 - 1	ReLu, Sigmoid	He, Xavier, Xavier	0.4	100 000	całe dane
				0.3	10 000	64

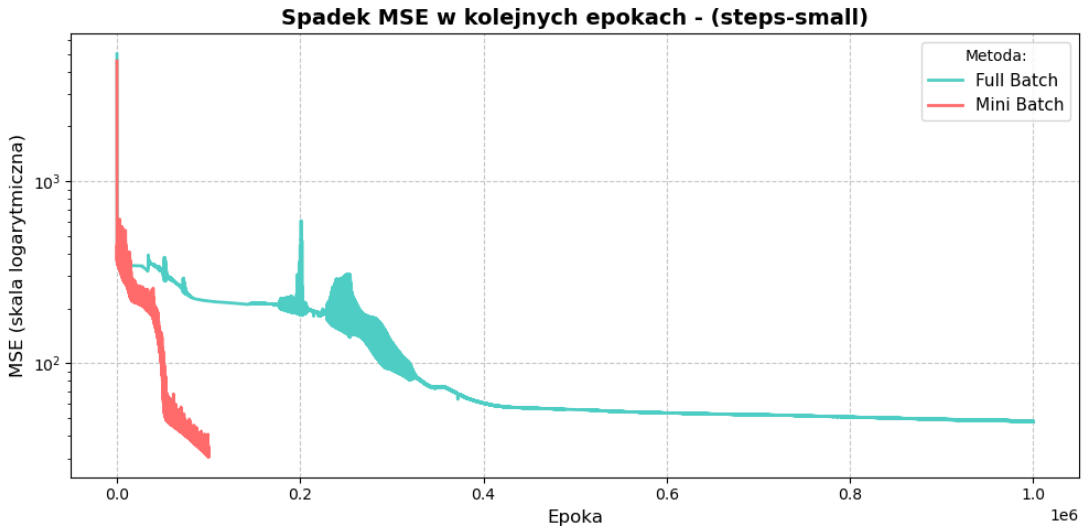
Tabela 2: Architektura modeli oraz parametry uczenia dla poszczególnych zbiorów.

Zbiór danych	Metoda	MSE na zbiorze treningowym	MSE na zbiorze testowym
square-simple	Full-Batch	1.156	1.281
	Mini-Batch	0.295	0.282
steps-small	Full-Batch	47.734	167.487
	Mini-Batch	30.997	100.098
multimodal-large	Full-Batch	172.081	181.895
	Mini-Batch	12.908	7.976

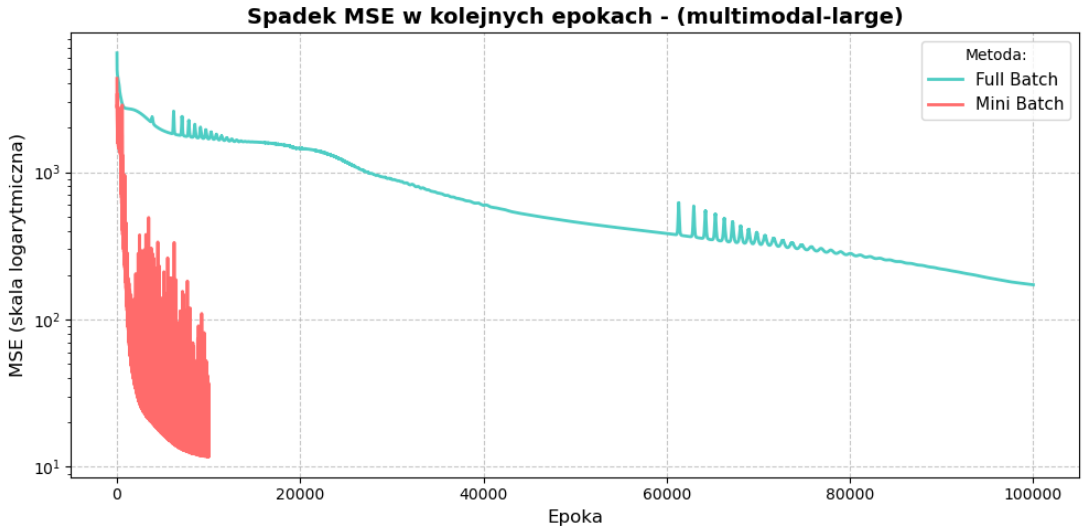
Tabela 3: Przybliżone nieznormalizowane finalne wartości funkcji straty dla konkretnych modeli i zbiorów danych.



Rysunek 4: Wartości MSE treningowego dla kolejnych epok dla metod Mini-Batch oraz Full-Batch dla zbioru square-simple.



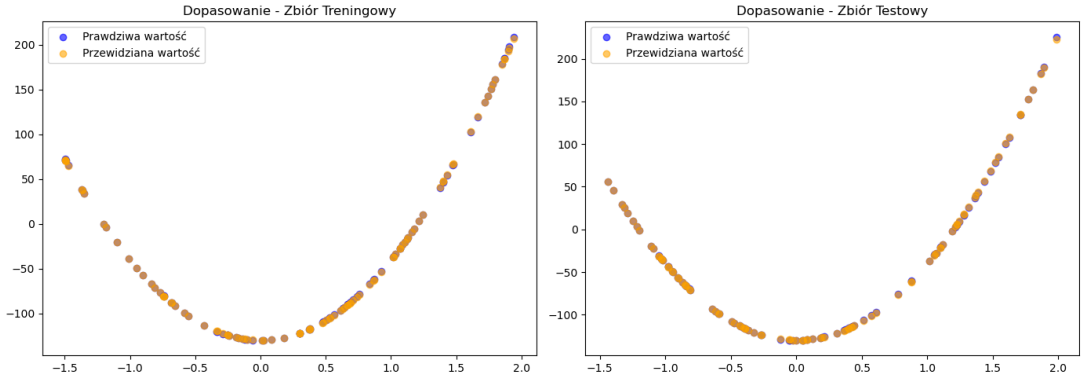
Rysunek 5: Wartości MSE treningowego dla kolejnych epok dla metod Mini-Batch oraz Full-Batch dla zbioru steps-small.



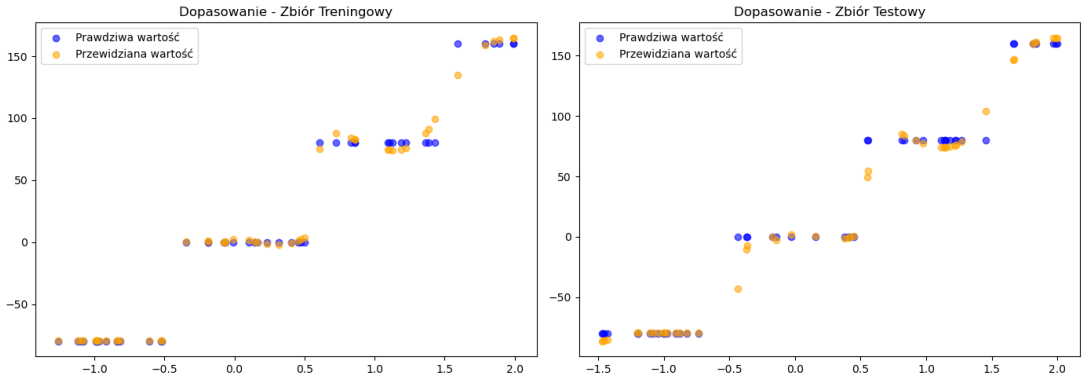
Rysunek 6: Wartości MSE treningowego dla kolejnych epok dla metod Mini-Batch oraz Full-Batch dla zbioru multimodal-large.

Dla każdego ze zbiorów danych aktualizacja wag w porcjach dawała niższe (czasem nawet znacząco) wartości funkcji straty w mniejszej liczbie epok. Do osiągnięcia niskich wyników MSE metoda Full-Batch potrzebowała większego tempa uczenia oraz większej liczby epok. Jest to spowodowane tym, że ta metoda częściej aktualizuje wagi oraz wprowadza szum, który zapobiega utykaniu algorytmu w minimach lokalnych i na płaskowyżach błędu.

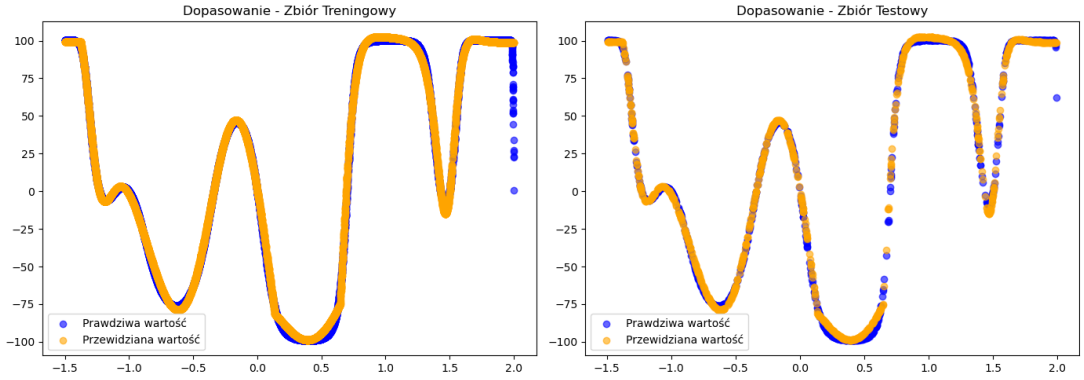
Na wykresach zaprezentowane jest działanie sieci dla modeli wywołanych metodą Mini-Batch dla każdego ze zbiorów. Przedstawione są rzeczywiste wartości danych oraz wartości przewidziane przez sieć. Wykres 7 przedstawia zbiór square-simple, wykres 8 zbiór steps-small, a wykres 9 zbiór multimodal-large.



Rysunek 7: Wartości rzeczywiste i przewidziane przez sieć dla danych square-simple metodą Mini-Batch.



Rysunek 8: Wartości rzeczywiste i przewidziane przez sieć dla danych steps-small metodą Mini-Batch.



Rysunek 9: Wartości rzeczywiste i przewidziane przez sieć dla danych multimodal-large metodą Mini-Batch.

Implementacja momentu i normalizacji gradientu

Na tym etapie zaimplementowane zostały dwa usprawnienia uczenia gradientowego sieci neuronowej: metoda momentu oraz normalizacja gradientu RMSProp. Metoda momentu przyspiesza zbieżność, zachowując "bezwładność"z poprzednich iteracji, co pomaga modelowi pokonywać płaskie obszary błędu. Z kolei RMSProp stabilizuje proces uczenia, dynamicznie dostosowując indywidualne tempo zmian dla każdej wagi na podstawie historii jej własnych gradientów. Teraz, inicjalizując model, można wybrać jedną z tych opcji optymalizacji, pamiętając, że wpływają one wyłącznie na sam matematyczny algorytm aktualizacji wag po ich wyliczeniu.

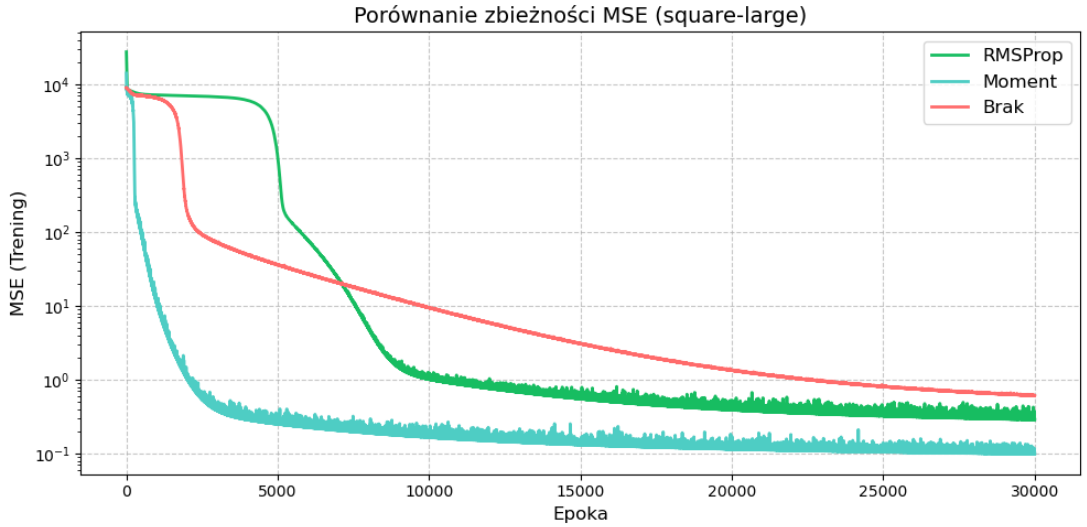
Na tym etapie analizowane były zbiory: square-large, steps-large oraz multimodal-large. Dla każdej z metod optymalizacji (momentu, RMSProp oraz klasycznego spadku gradientowego) dobrano optymalne architektury sieci, współczynniki uczenia oraz liczbę epok. Szczegółowe zestawienie parametrów znajduje się w tabeli 4.

Zbiór danych	Architektura	Funkcje aktywacji	Inicjalizacje wag	Wielkość porcji	Szybkość uczenia	Liczba epok	Metoda optymalizacji
square-large	1-10-5-1	Sigmoid, Sigmoid	Xavier, Xavier, Xavier	64	0.1	30 000	Brak
					0.1		Moment
					0.0001		RMSProp
steps-large	1-10-10-1	Sigmoid, Sigmoid	Xavier, Xavier, Xavier	32	0.8	18 000	Brak
					0.1		Moment
					0.001		RMSProp
multimodal-large	1-12-12-1	Sigmoid, Sigmoid	Xavier, Xavier, Xavier	64	0.3	15 000	Brak
					0.1		Moment
					0.001		RMSProp

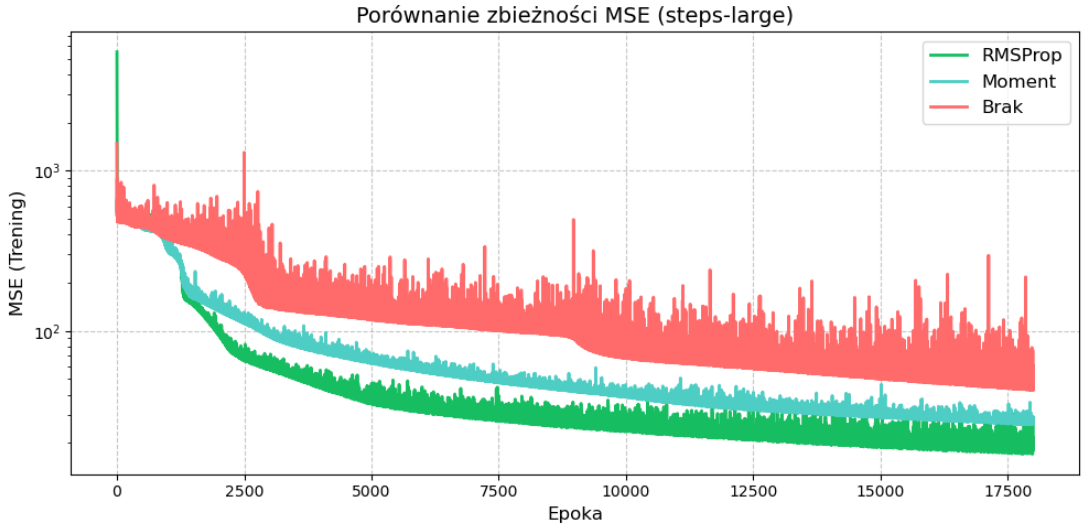
Tabela 4: Architektura modeli oraz parametry uczenia dla poszczególnych zbiorów i metod optymalizacji.

Z eksperymentów wynika, że przy użyciu metody RMSProp współczynnik uczenia musi być znacząco niższy niż w przypadku pozostałych algorytmów. Wynika to z faktu, że RMSProp normalizuje wielkość gradientów - jeśli bazowy learning rate jest zbyt wysoki, każda aktualizacja staje się zbyt agresywna względem aktualnej skali błędu, co prowadzi do oscylacji i uniemożliwia stabilną zbieżność.

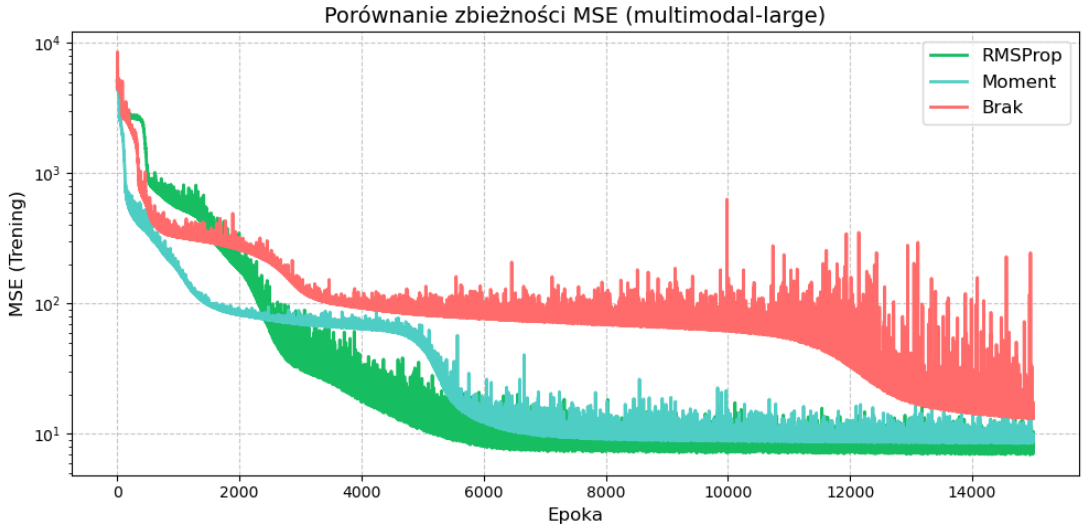
Analizując wykresy zbieżności (10, 11, 12), można zauważyć, że metoda momentu zbiega najszybciej. Dzieje się tak, ponieważ akumuluje ona prędkość od pierwszych iteracji, co pozwala na agresywne schodzenie po zboczu funkcji straty. Z kolei RMSProp potrzebuje więcej czasu na początkową fazę nauki, gdyż algorytm musi najpierw zgromadzić odpowiednią historię kwadratów gradientów, aby poprawnie wyskalować wielkość kroków. Jednak po tej fazie wstępnej, dzięki precyzyjnemu dopasowaniu tempa uczenia, RMSProp często osiąga lepsze wyniki końcowe niż standardowy model, a niekiedy przewyższa skutecznością metodę momentu w końcowej fazie optymalizacji.



Rysunek 10: Wartości MSE treningowego dla kolejnych epok dla metod optymalizacji uczenia gradientowego dla zbioru square-large.



Rysunek 11: Wartości MSE treningowego dla kolejnych epok dla metod optymalizacji uczenia gradientowego dla zbioru steps-large.

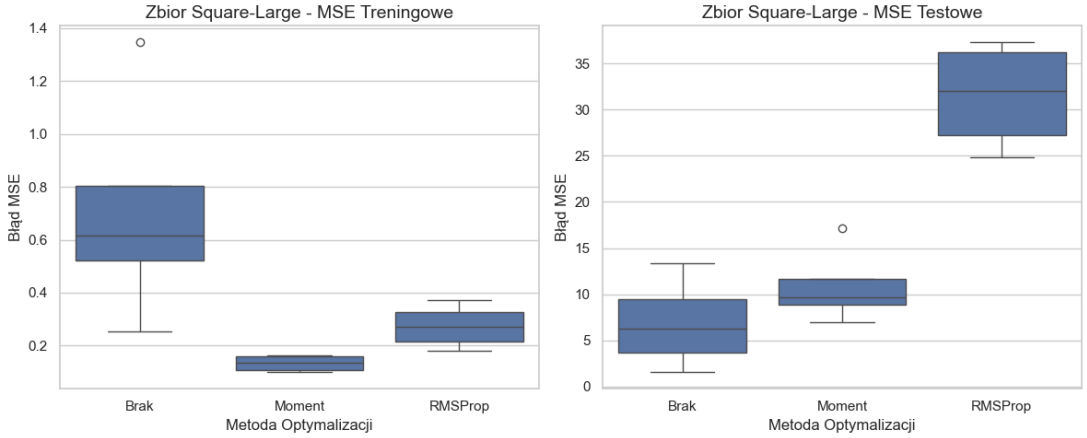


Rysunek 12: Wartości MSE treningowego dla kolejnych epok dla metod optymalizacji uczenia gradientowego dla zbioru multimodal-large.

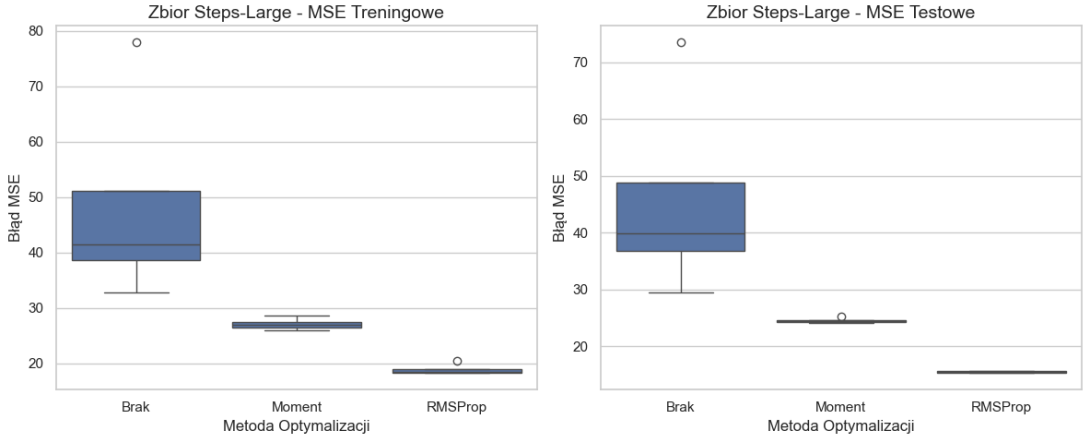
Każdy model został przetestowany 4 razy, aby upewnić się, że wyniki nie są zależne od danego losowego wywołania przy jednoczesnym ograniczeniu czasu wywołania. Otrzymane średnie wyniki wraz z odchyleniem standardowym MSE dla każdego ze zbiorów i metod optymalizacji umieszczone są w tabeli 5 oraz na wykresach 13, 14, 15. Analiza wyników wskazuje, że zastosowanie metod optymalizacji znacząco usprawnia proces uczenia. Dla większości badanych zbiorów pozwalają one na osiągnięcie wyraźnie mniejszego błędu treningowego oraz gwarantują znacznie wyższą stabilność modelu, co objawia się drastycznym spadkiem odchylenia standardowego względem modelu bazowego. Wybór najlepszej metody jest silnie zależny od specyfiki danych: na zbiorze steps-large najmniejszy błąd wygenerował algorytm RMSProp, podczas gdy dla multimodal-large minimalnie skuteczniejsza okazała się metoda momentu. Szczególnie interesującym przypadkiem jest jednak zbiór square-large. Choć optymalizatory błyskawicznie zminimalizowały na nim błąd treningowy, to najniższy błąd na zbiorze testowym osiągnął model pozbawiony tych usprawnień. Zjawisko to może być objawem przeuczenia - algorytmy przyspieszające uczenie tak dokładnie dopasowały wagi do danych treningowych, że sieć częściowo utraciła zdolność poprawnej generalizacji na nowych danych.

Zbiór danych	Metoda optymalizacji	MSE Treningowe		MSE Testowe	
		Średnia	Odch. std.	Średnia	Odch. std.
square-large	Brak	0.708	0.458	6.849	5.108
	Moment	0.133	0.033	10.861	4.367
	RMSProp	0.273	0.085	31.547	6.037
steps-large	Brak	48.411	20.116	45.692	19.147
	Moment	27.091	1.123	24.547	0.499
	RMSProp	18.865	1.0158	15.547	0.194
multimodal-large	Brak	23.421	9.053	19.484	10.068
	Moment	8.383	0.377	4.057	0.483
	RMSProp	8.431	2.328	4.810	3.108

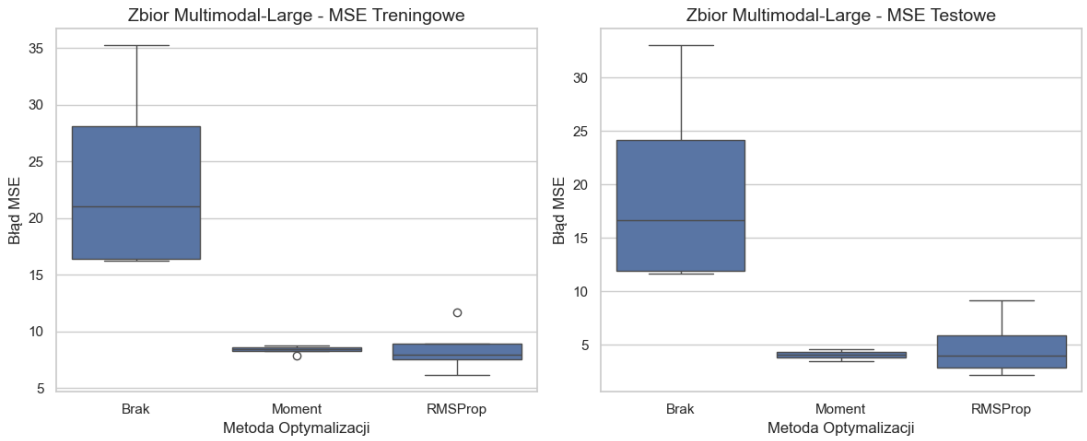
Tabela 5: Średnie przybliżone nieznormalizowane finalne wartości funkcji straty oraz ich odchylenia standardowe dla konkretnych metod optymalizacji i zbiorów danych.



Rysunek 13: Rozkład MSE treningowego i testowego dla kolejnych metod optymalizacji dla zbioru square-large.



Rysunek 14: Rozkład MSE treningowego i testowego dla kolejnych metod optymalizacji dla zbioru steps-large.



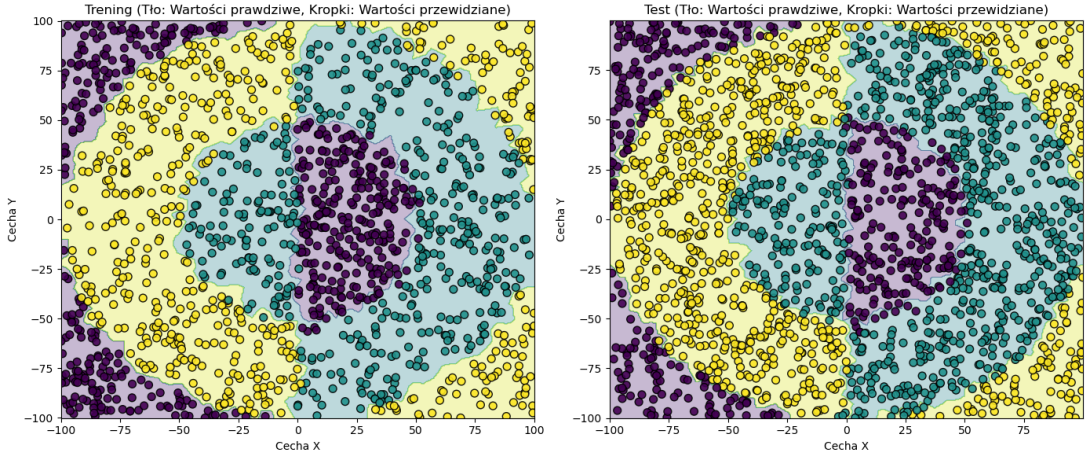
Rysunek 15: Rozkład MSE treningowego i testowego dla kolejnych metod optymalizacji dla zbioru multimodal-large.

Rozwiązywanie zadania klasyfikacji

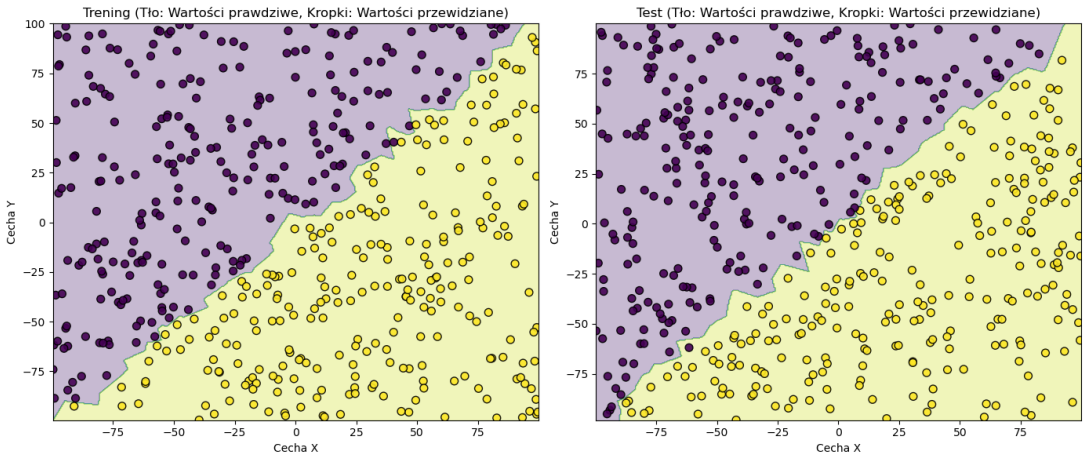
Na tym etapie prowadzone były eksperymenty na zadaniach klasyfikacyjnych ze zbiorów rings3-regular, easy i xor3. W tym celu zaimplementowane zostały trzy kluczowe mechanizmy. Kodowanie one-hot wykorzystano do zamiany etykiet na wektory binarne, co pozwala sieci traktować każdą klasę jako niezależną kategorię. Na ostatniej warstwie modelu wprowadzono funkcję aktywacji softmax, która normalizuje surowe wyniki sieci, przekształcając je w czytelny rozkład prawdopodobieństw sumujący się do jedynki. Z kolei jako funkcję błędu zastosowano entropię krzyżową (cross-entropy), która precyzyjnie mierzy odległość między przewidywaniami modelu a rzeczywistymi etykietami, skutecznie karząc sieć za błędne i zbyt pewne siebie decyzje. Dla każdego zbioru porównane zostały wyniki uzyskane dla modeli korzystających z funkcji softmax oraz dla modeli, które na ostatniej warstwie używają funkcji sigmoid. Używane architektury dla konkretnych zbiorów umieszczone są w tabeli 6. Na wykresach 16, 17, 18 widoczne są prawdziwe i przewidziane klasy przez modele używające funkcji softmax dla zbiorów treningowego i testowego.

Zbiór danych	Architektura	Funkcje aktywacji	Inicjalizacje wag	Wielkość porcji	Szybkość uczenia	Liczba epok	Metoda optymalizacji
rings3-regular	2-10-5-N	Sigmoid, Sigmoid	Xavier, Xavier, Xavier	64	0.1	2 000	Moment
easy	2-10-5-N	Sigmoid, Sigmoid	Xavier, Xavier, Xavier	64	0.1	1 000	Moment
xor3	2-10-5-N	Sigmoid, Sigmoid	Xavier, Xavier, Xavier	16	0.1	8 000	Moment

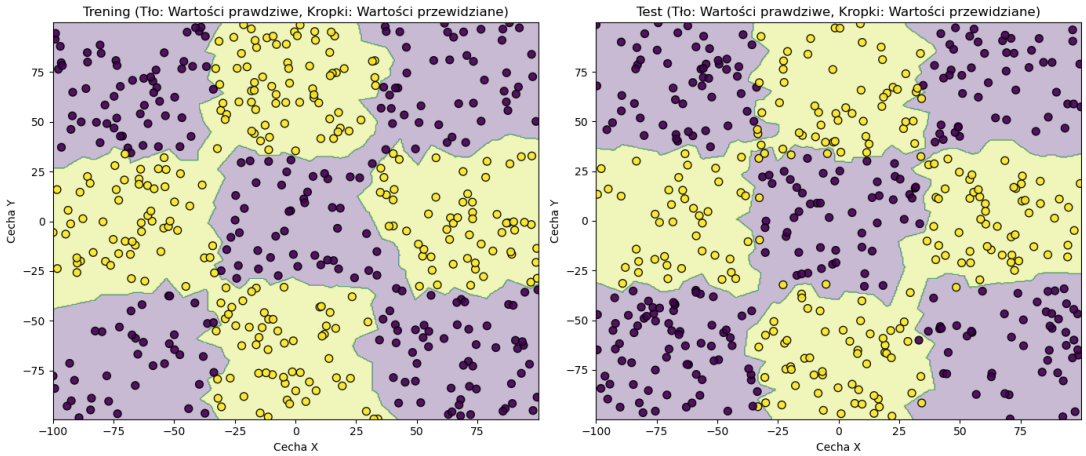
Tabela 6: Architektura modeli oraz parametry uczenia dla poszczególnych zbiorów, gdzie N oznacza liczbę klas zbioru.



Rysunek 16: Prawdziwe i przewidziane klasy przez model używający funkcji softmax dla treningowego i testowego zbioru rings3-regular. Kolor tła oznacza prawdziwe wartości zbioru, a kolor kropek to wartości przewidziane przez model.



Rysunek 17: Prawdziwe i przewidziane klasy przez model używający funkcji softmax dla treningowego i testowego zbioru easy. Kolor tła oznacza prawdziwe wartości zbioru, a kolor kropek to wartości przewidziane przez model.

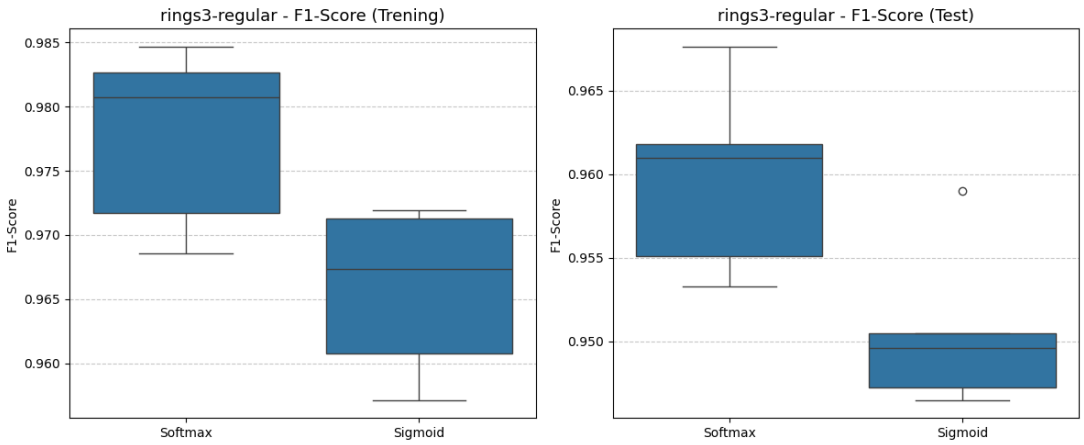


Rysunek 18: Prawdziwe i przewidziane klasy przez model używający funkcji softmax dla treningowego i testowego zbioru xor3. Kolor tła oznacza prawdziwe wartości zbioru, a kolor kropek to wartości przewidziane przez model.

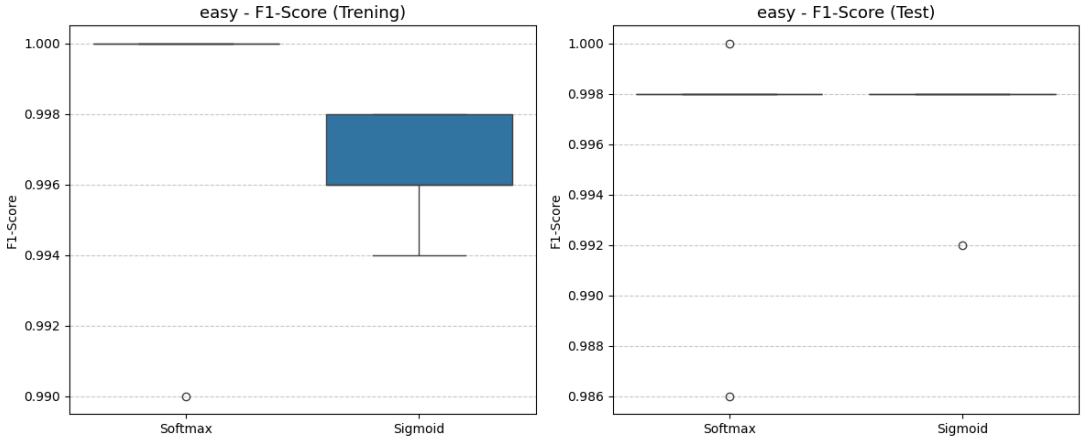
Na tym etapie do sprawdzenia poprawności klasyfikacji używana była metryka F1-Score. Każdy model został przetestowany 5 razy, aby upewnić się, że wyniki nie są zależne od danego losowego wywołania przy jednoczesnym ograniczeniu czasu wywołania. Otrzymane średnie wyniki wraz z odchyleniem standardowym F1-Score dla każdego ze zbiorów i funkcji aktywacji umieszczone są w tabeli 7 oraz na wykresach 19, 20, 21. Wykresy 22, 23, 24 porównują prędkość zbieżności modeli używających funkcji softmax oraz sigmoid dla jednego z wywołań. Od razu widać, że dodanie funkcji softmax poprawia wyniki modelu - dla tych samych wartości innych parametrów, wartości F1-Score są średnio wyższe. Można także zauważyć, że zbieżność modelu używającego funkcji softmax następuje szybciej dla wszystkich zbiorów danych. Potwierdza to, że funkcja softmax jest niezbędna w zadaniach klasyfikacji. Normalizuje ona surowe wyjścia sieci do postaci prawidłowego rozkładu prawdopodobieństw, co w połączeniu z entropią krzyżową tworzy optymalne i stabilne środowisko do aktualizacji wag, pozwalając modelowi na pewniejsze odseparowanie od siebie poszczególnych klas.

Zbiór danych	Ostatnia funkcja aktywacji	F1-Score treningowe		F1-Score testowe	
		Średnia	Odch. std.	Średnia	Odch. std.
rings3-regular	Sigmoid	0.966	0.007	0.951	0.005
	Softmax	0.978	0.007	0.960	0.006
easy	Sigmoid	0.996	0.002	0.997	0.003
	Softmax	0.998	0.004	0.996	0.006
xor3	Sigmoid	0.992	0.013	0.941	0.017
	Softmax	0.992	0.005	0.943	0.015

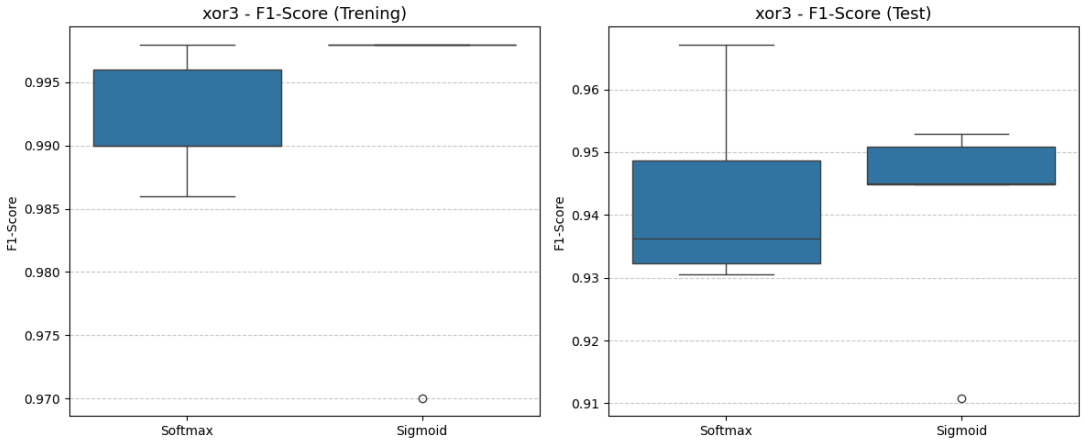
Tabela 7: Średnie finalne wartości metryki F1-Score oraz ich odchylenia standardowe dla konkretnych modeli i zbiorów danych.



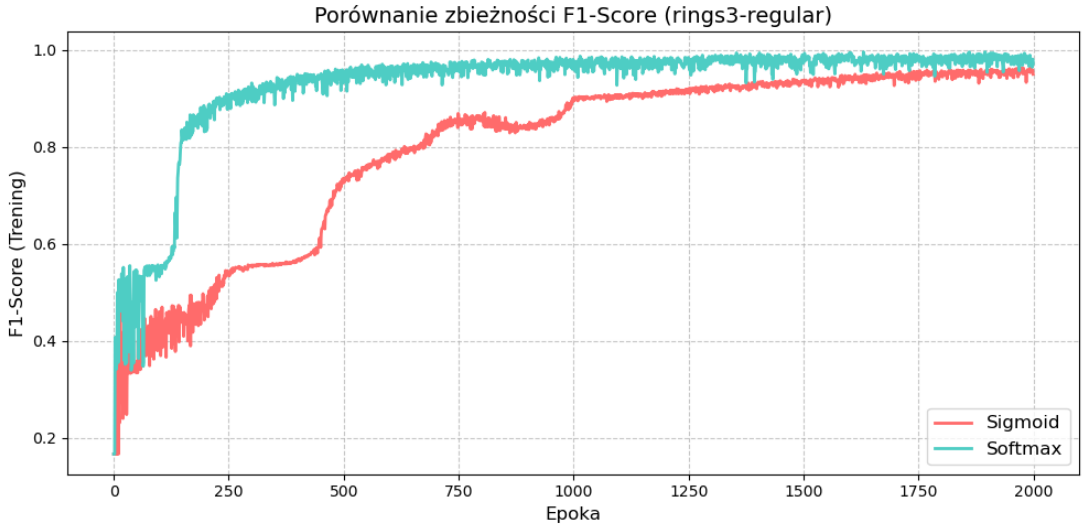
Rysunek 19: Rozkład F1-Score treningowego i testowego dla użytej funkcji softmax lub sigmoid dla zbioru rings3-regular.



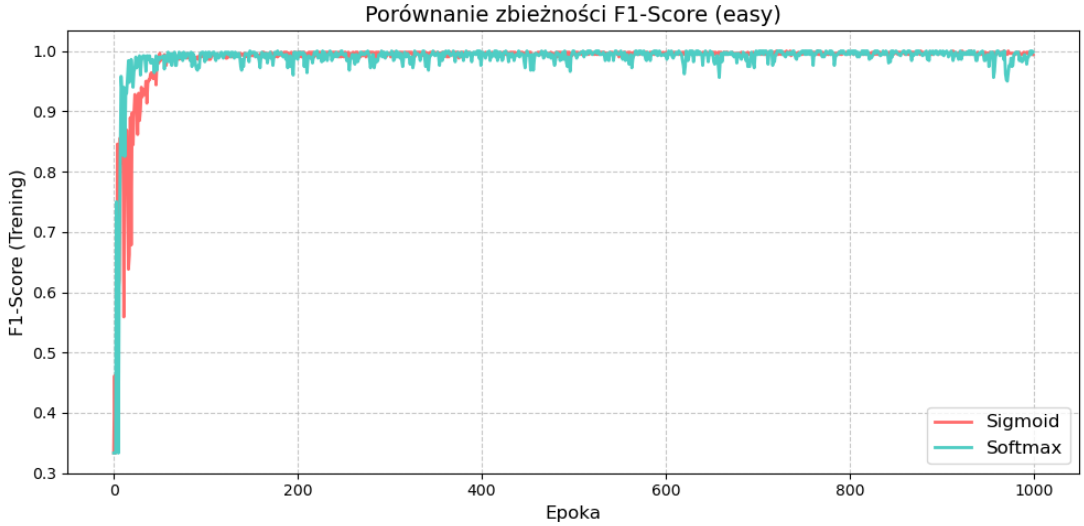
Rysunek 20: Rozkład F1-Score treningowego i testowego dla użytej funkcji softmax lub sigmoid dla zbioru easy.



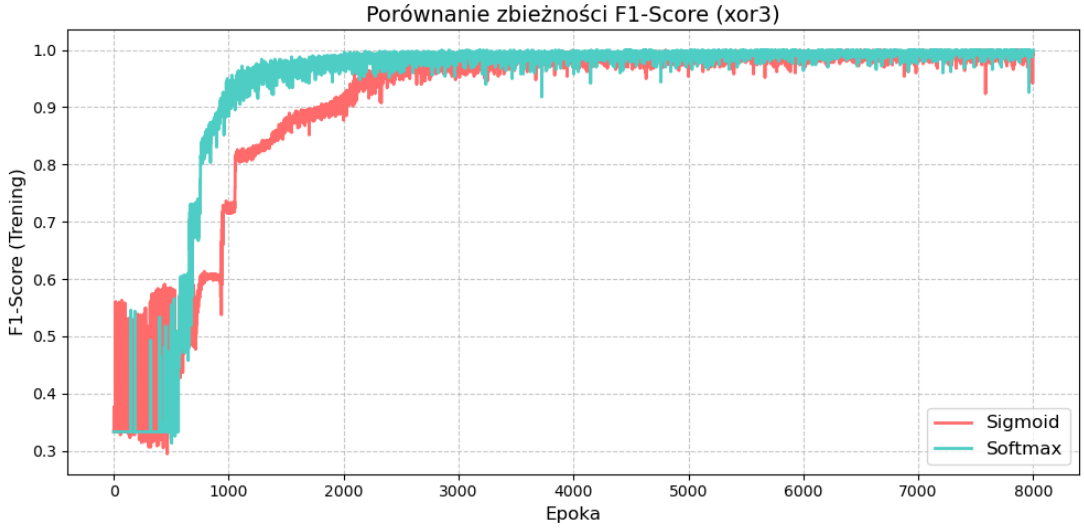
Rysunek 21: Rozkład F1-Score treningowego i testowego dla użytej funkcji softmax lub sigmoid dla zbioru xor3.



Rysunek 22: Wartości F1-Score treningowego dla kolejnych epok dla modelu z aktywacją softmax oraz sigmoid dla zbioru rings3-regular.



Rysunek 23: Wartości F1-Score treningowego dla kolejnych epok dla modelu z aktywacją softmax oraz sigmoid dla zbioru easy.



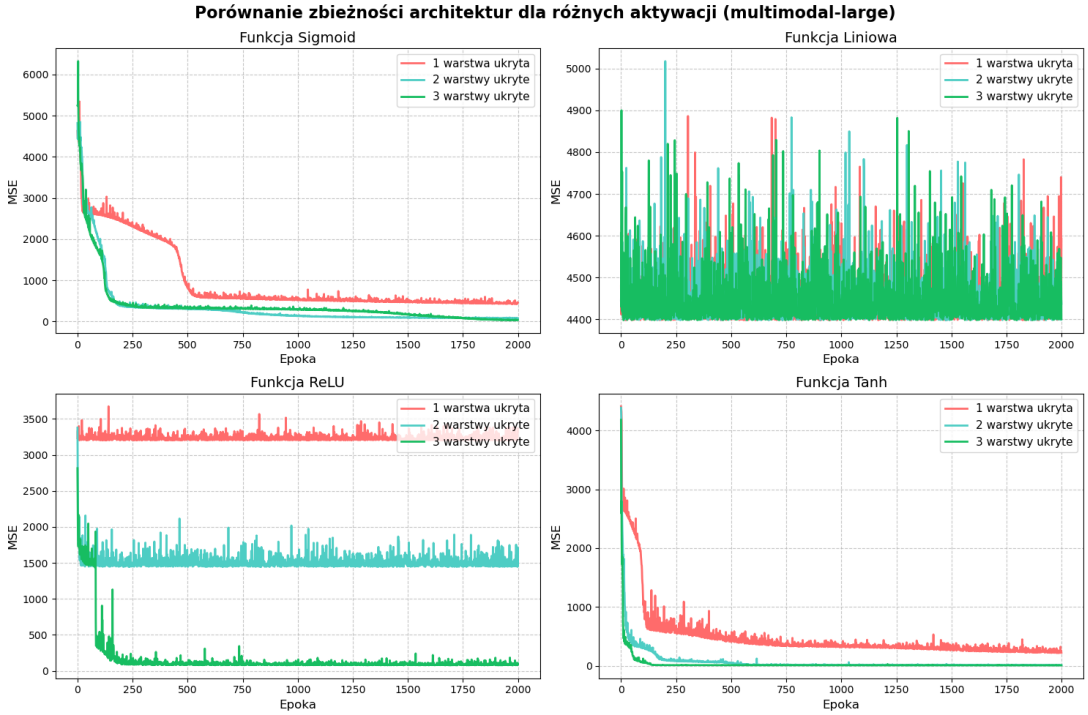
Rysunek 24: Wartości F1-Score treningowego dla kolejnych epok dla modelu z aktywacją softmax oraz sigmoid dla zbioru xor3.

Testowanie różnych funkcji aktywacji

W ramach tego etapu zaimplementowano brakujące funkcje aktywacji, co pozwoliło na testowanie sieci z wykorzystaniem czterech wariantów: Sigmoid, ReLU, funkcji liniowej oraz Tanh. Każda z nich posiada unikalne właściwości: Sigmoid spłaszcza sygnał do przedziału (0, 1), ReLU zeruje wartości ujemne, przyspieszając obliczenia, funkcja liniowa przekazuje sygnał bez zmian, natomiast tangens hiperboliczny centruje wyniki wokół zera, skalując je do przedziału (-1, 1). Aby zapewnić spójne warunki eksperymentu, wszystkie modele trenowano, wykorzystując optymalizator momentum ze współczynnikiem 0.9 oraz szybkością uczenia rzędu 0.1. Proces nauki trwał 2000 epok, a dane podawano w paczkach o rozmiarze 64 próbek. Dodatkowo, w przypadku zadań klasyfikacyjnych, warstwę wyjściową sieci zawsze wieńczyła funkcja Softmax.

Do pierwszego testowania wpływu różnych funkcji aktywacji, różnej liczby warstw i różnej liczby neuronów został użyty zbiór multimodal-large. Wywołano 36 różnych architektur z różnymi funkcjami aktywacji. Ich parametry i wyniki są umieszczone w tabeli 8. Na wykresie 26 porównana jest zbieżność modeli dla różnej liczby warstw (każda z 10 neuronami) dla różnych funkcji aktywacji. Na wykresach 25 porównana jest zbieżność modeli dla różnych funkcji aktywacji. Na wykresie 27 widoczne jest pełne porównanie wyników MSE dla wszystkich wywołanych modeli. Na kolejnych wykresach przedstawione jest dopasowanie modelu dającego najlepsze wyniki dla każdej z dostępnych funkcji aktywacji. Wykres 28 to model z funkcją Sigmoid i trzema warstwami po 20 neuronów. Wykres 29 to model z funkcją liniową i jedną warstwą z 20 neuronami. Wykres 30 to model z funkcją ReLu i trzema warstwami po 20 neuronów. Wykres 31 to model z funkcją Tanh i trzema warstwami po 10 neuronów.

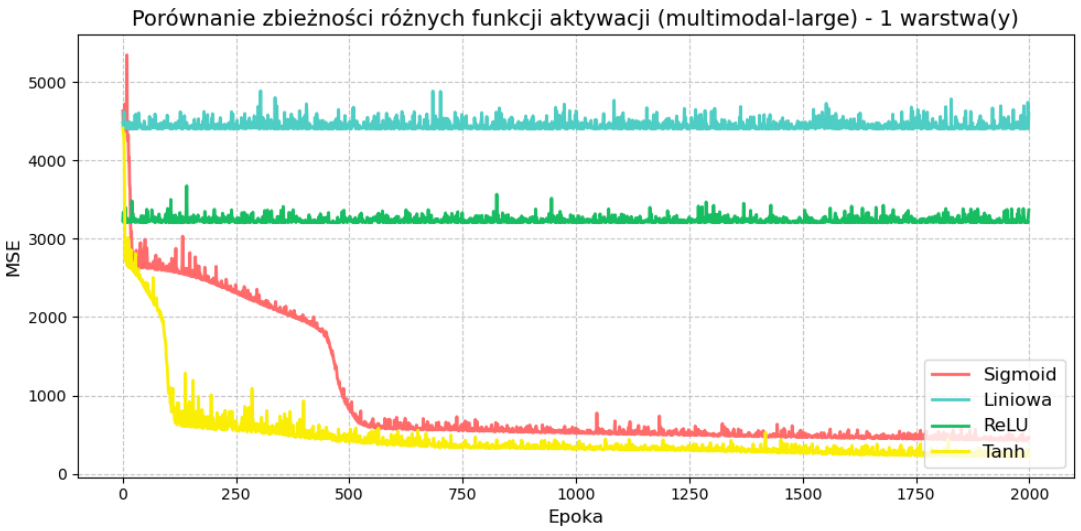
Po tych testach można było zauważyć, że zwykle im model miał więcej warstw bądź neuronów, tym wyższe wartości MSE uzyskiwał. Zdarzało się jednak, że zmiana z 10 na 20 neuronów (głównie w modelach z jedną warstwą) pogarszała wynik końcowy. Porównując zbieżność różnych modeli, można zauważyć, że funkcja Tanh w każdym z przypadków zbiegała w najmniejszej liczbie epok. Zwykle zbieżność niewiele wolniej osiągał również model z aktywacją Sigmoid. Model z funkcją liniową nie zbiegał wcale, natomiast dla funkcji ReLu zbiegł jedynie model z trzema warstwami. Porównując natomiast zbieżność w zależności od liczby warstw ukrytych, wyłania się prosta zależność dla każdej z funkcji aktywacji - im więcej warstw, tym szybsza zbieżność.



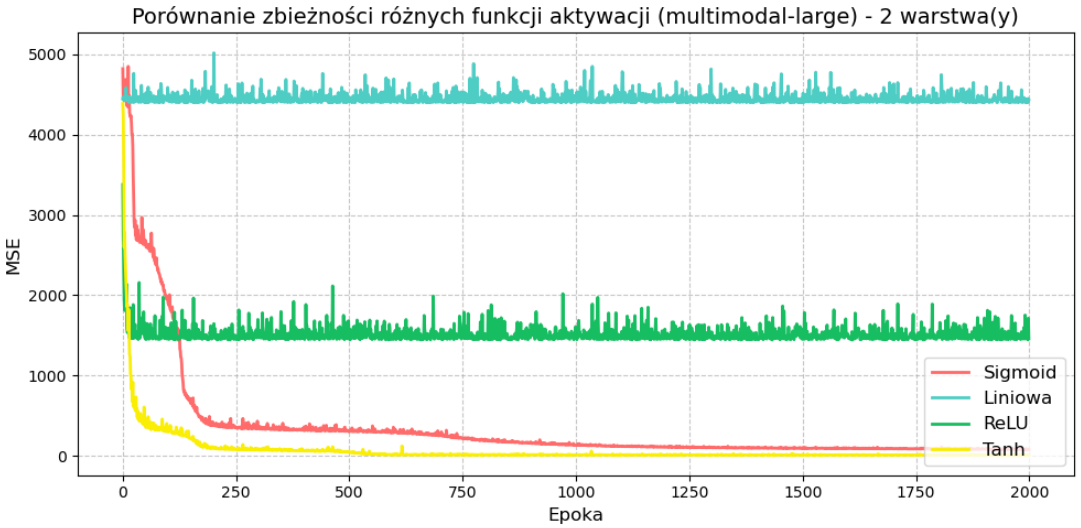
Rysunek 26: Wartości MSE treningowego dla kolejnych epok dla modeli z różnymi funkcjami aktywacji dla modeli z różną liczbą warstw (z 10 neuronami na warstwie).

Architektura	Funkcja aktywacji	MSE treningowe	MSE testowe
1-5-1	Sigmoid	431.056	432.106
1-10-1		453.419	446.524
1-20-1		515.833	525.651
1-5-1	Liniowa	4430.807	4466.895
1-10-1		4592.914	4638.621
1-20-1		4401.094	4435.851
1-5-1	ReLu	2732.725	2751.726
1-10-1		3365.675	3422.317
1-20-1		3306.285	3366.626
1-5-1	Tanh	356.784	360.516
1-10-1		219.969	232.405
1-20-1		291.099	299.667
1-5-5-1	Sigmoid	271.575	287.836
1-10-10-1		78.953	70.776
1-20-20-1		105.272	101.586
1-5-5-1	Liniowa	4405.448	4441.197
1-10-10-1		4437.419	4477.496
1-20-20-1		4414.949	4447.642
1-5-5-1	ReLu	2548.984	2643.103
1-10-10-1		1459.122	1499.155
1-20-20-1		27.740	24.432
1-5-5-1	Tanh	16.319	11.447
1-10-10-1		9.631	5.897
1-20-20-1		8.451	5.203
1-5-5-5-1	Sigmoid	112.509	106.797
1-10-10-10-1		35.221	30.599
1-20-20-20-1		32.789	29.248
1-5-5-5-1	Liniowa	4516.324	4559.775
1-10-10-10-1		4400.590	4436.906
1-20-20-20-1		4445.215	4485.441
1-5-5-5-1	ReLu	5228.831	5269.080
1-10-10-10-1		89.324	89.258
1-20-20-20-1		10.977	7.344
1-5-5-5-1	Tanh	8.752	5.238
1-10-10-10-1		5.946	1.843
1-20-20-20-1		5.956	1.863

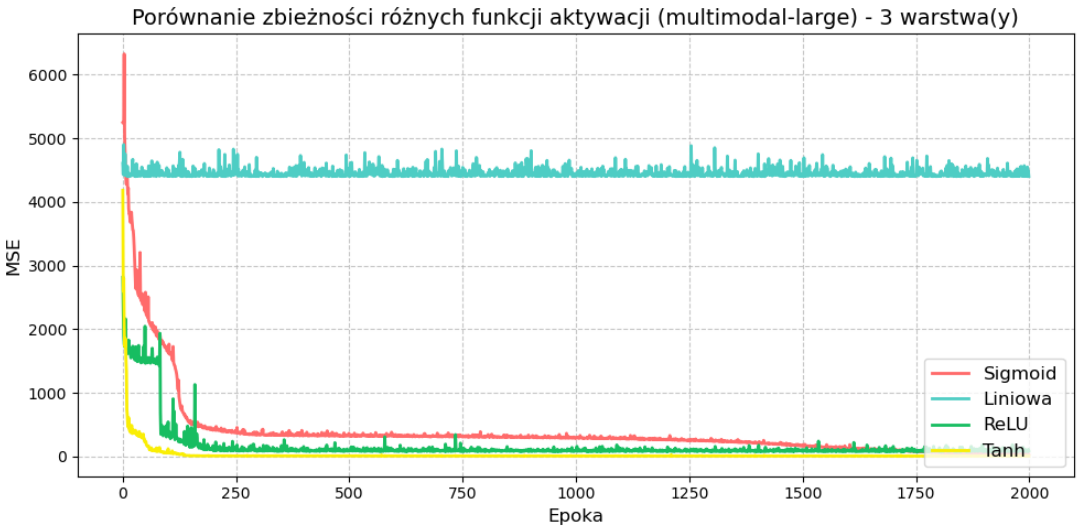
Tabela 8: Przybliżone finalne wartości funkcji straty dla konkretnych modeli i funkcji aktywacji.



(a) MSE dla modelu z 1 warstwą z 10 neuronami.



(b) MSE dla modelu z 2 warstwami po 10 neuronów.

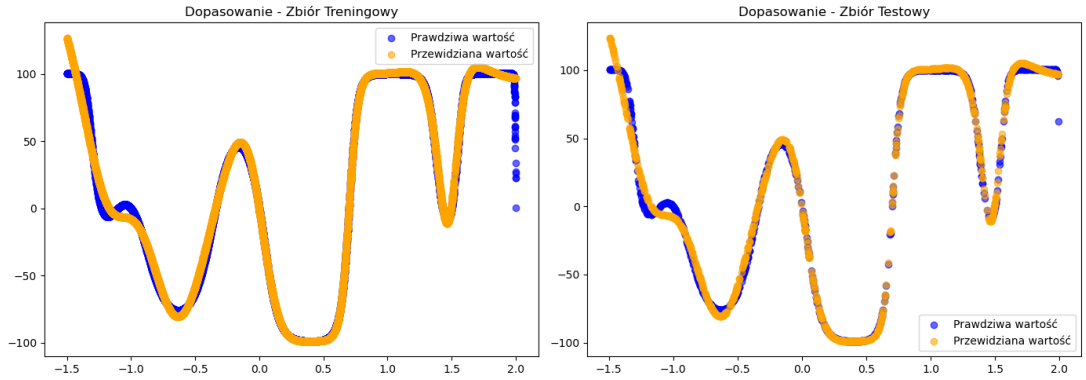


(c) MSE dla modelu z 3 warstwami po 10 neuronów.

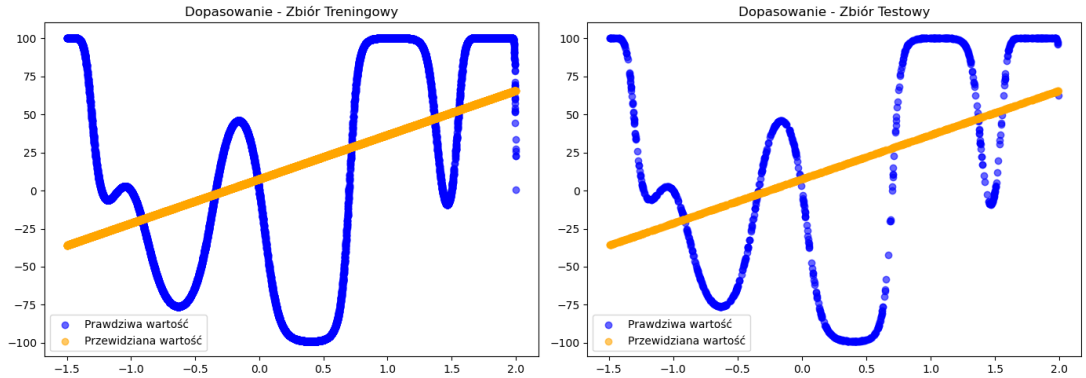
Rysunek 25: Wartości MSE treningowego dla kolejnych epok dla modeli z różnymi funkcjami aktywacji.



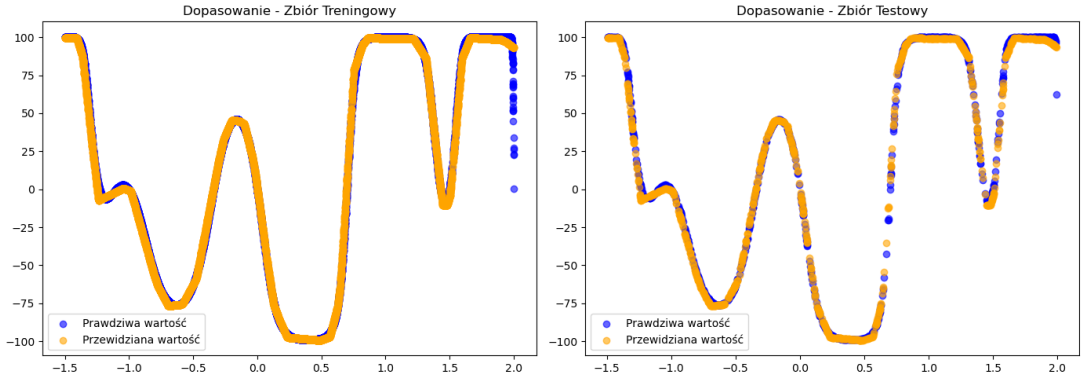
Rysunek 27: Wartości MSE treningowego dla modeli z różnymi architekturami i funkcjami aktywacji.



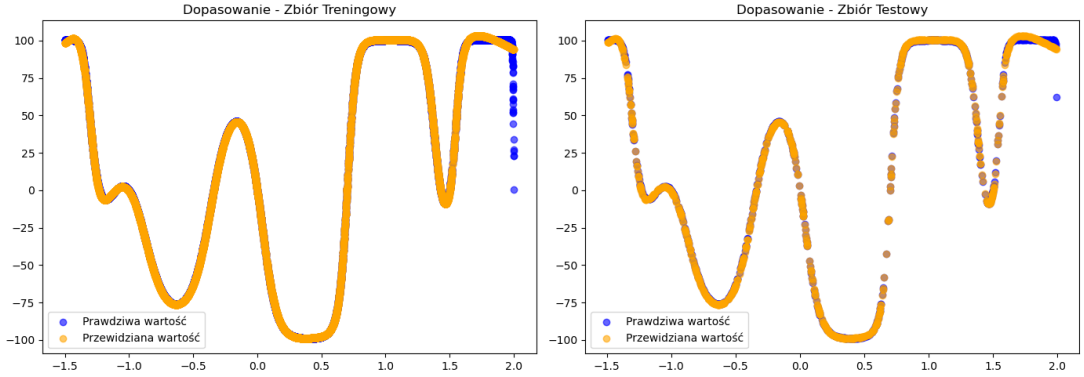
Rysunek 28: Wartości rzeczywiste i przewidziane przez sieć dla danych multimodal-large z architekturą z 3 warstwami po 20 neuronów i funkcją aktywacji Sigmoid.



Rysunek 29: Wartości rzeczywiste i przewidziane przez sieć dla danych multimodal-large z architekturą z 1 warstwą z 20 neuronami i funkcją aktywacji liniową.



Rysunek 30: Wartości rzeczywiste i przewidziane przez sieć dla danych multimodal-large z architekturą z 3 warstwami po 20 neuronów i funkcją aktywacji ReLu.

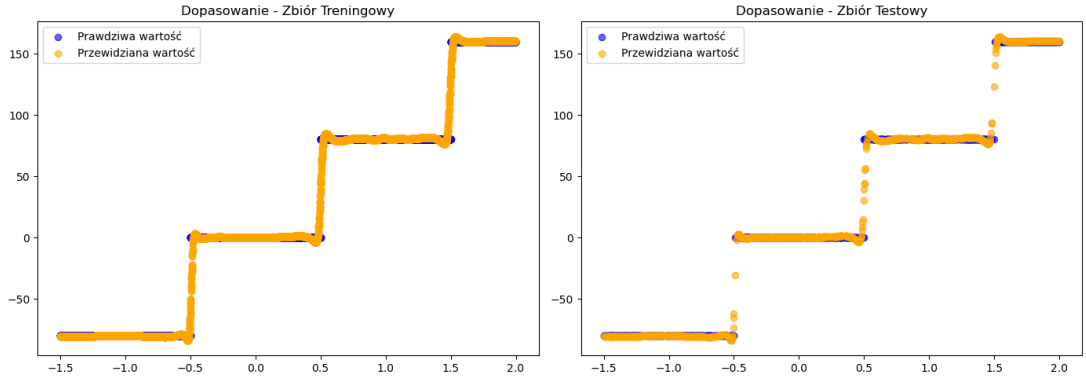


Rysunek 31: Wartości rzeczywiste i przewidziane przez sieć dla danych multimodal-large z architekturą z 3 warstwami po 10 neuronów i funkcją aktywacji Tanh.

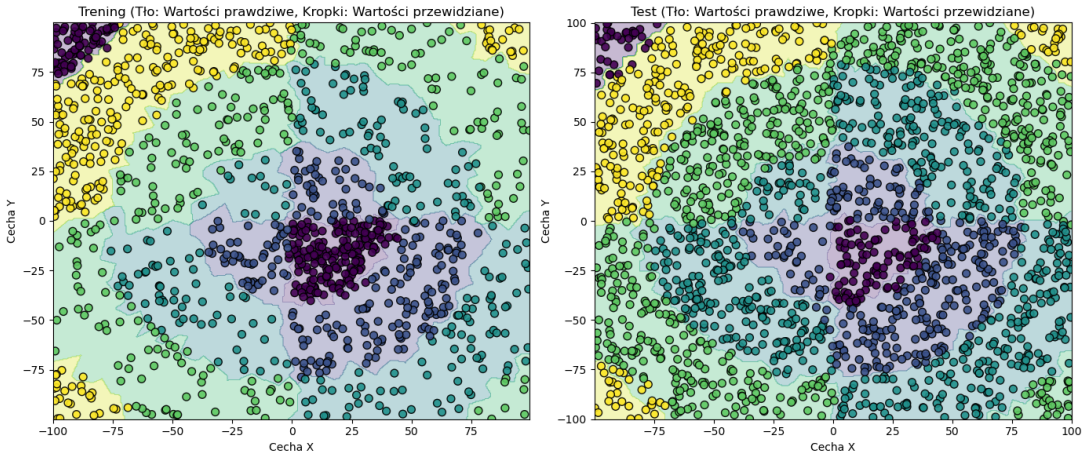
Dla zbiorów danych steps-large, rings5-regular i rings3-regular zostały wywołane 4 architektury dające najlepsze wyniki MSE. Wybrane architektury oraz wyniki danej metryki dla każdego ze zbiorów są widoczne w tabeli 9. Wyjątkowo model dla danych steps-large został wywołany dla 6000 epok zamiast 2000. Na wykresach widoczne jest dopasowanie modelu dla architektury dającej najlepszy wynik na kolejnych zbiorach testowych. Zbiór steps-large jest widoczny na wykresie 32, rings5-regular na wykresie 33, a rings3-regular na wykresie 34.

Zbiór danych	Architektura	Funkcja aktywacji	MSE/F1 treningowe	MSE/F1 testowe
steps-large	1-10-5-1	Sigmoid	325.530	319.537
	1-10-5-1	Tanh	43.353	40.849
	1-10-10-5-1	Sigmoid	121.914	116.864
	1-10-10-5-1	Tanh	18.034	17.104
rings5-regular	2-10-5-N	Sigmoid	0.945	0.912
	2-10-5-N	Tanh	0.908	0.825
	2-10-10-5-N	Sigmoid	0.968	0.924
	2-10-10-5-N	Tanh	0.933	0.876
rings3-regular	2-10-5-N	Sigmoid	0.979	0.956
	2-10-5-N	Tanh	0.914	0.895
	2-10-10-5-N	Sigmoid	0.982	0.957
	2-10-10-5-N	Tanh	0.957	0.940

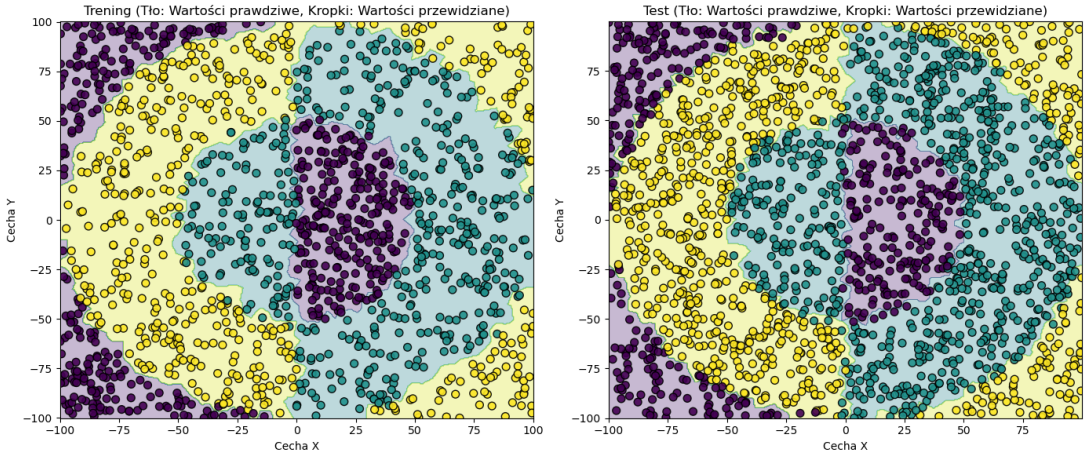
Tabela 9: Architektura modeli oraz wartości metryk dla poszczególnych zbiorów, gdzie N oznacza liczbę klas zbioru. Dla zbioru steps-large liczona była wartość MSE, a dla pozostałych F1.



Rysunek 32: Wartości rzeczywiste i przewidziane przez sieć dla danych steps-large z architekturą z 3 warstwami i funkcją aktywacji Tanh.



Rysunek 33: Prawdziwe i przewidziane klasy przez model dla treningowego i testowego zbioru rings5-regular z architekturą z 3 warstwami i funkcją aktywacji Sigmoid.



Rysunek 34: Prawdziwe i przewidziane klasy przez model dla treningowego i testowego zbioru rings3-regular z architekturą z 3 warstwami i funkcją aktywacji Sigmoid.

Aby zapewnić powtarzalność wyników, część eksperymentów została powtórzona 3 razy. Dla każdego zbioru danych przetestowano modele o różnej głębokości (1, 2 lub 3 warstwy ukryte po 10 neuronów), sprawdzając dla każdej z tych architektur cztery różne funkcje aktywacji. Podsumowanie średnich wyników metryk oraz ich odchyleń standardowych znajduje się w tabelach 10 i 11 oraz na wykresach 35, 36, 37, 38.

Ze wszystkich wykonanych eksperymentów można wnioskować, że we wszystkich przypadkach najlepsze i najbardziej stabilne wyniki dawał tangens hiperboliczny, gdzie już dwie warstwy wystarczały do uzyskania zadowalających rezultatów. Wynika to z działania funkcji, które zapobiega oscylacjom i znacznie przyspiesza zbieżność algorytmu optymalizacyjnego. Zwykle nieco gorsze, ale wciąż poprawne wyniki dawała funkcja Sigmoid, chociaż tutaj zazwyczaj

potrzebne były aż trzy warstwy ukryte. Jest to spowodowane tym, że Sigmoid ma wolniejszą aktualizację wag, a dodatkowo jest silniej narażony na zjawisko znikającego gradientu, przez co płytsze sieci miały problem z poprawnym modelowaniem nieliniowości. Wyniki funkcji ReLU miały zdecydowanie największe odchylenie standardowe - obserwowano rezultaty dobre, jak i bardzo złe. Mogło to być spowodowane działaniem funkcji, która mogła zerować wagę neuronu i drastycznie obniżać skuteczność całej sieci. Co ciekawe, dodanie trzeciej warstwy dodatkowo obniżało wynik w porównaniu z modelem z dwoma warstwami. Liniowa funkcja aktywacji w każdym przypadku dawała znacząco gorsze wyniki, czego w pełni można się było spodziewać. Złożenie wielu warstw liniowych jest matematycznie równoważne pojedynczej warstwie liniowej. Model pozbawiony funkcji nieliniowych traci możliwość wyznaczania skomplikowanych granic decyzyjnych, przez co nie jest w stanie rozwiązać problemów nieliniowo separowalnych.

Zbiór danych	Aktywacja	1 warstwa		2 warstwy		3 warstwy	
		Średnia	Odch. std.	Średnia	Odch. std.	Średnia	Odch. std.
multimodal-large	Liniowa	4422.622	16.532	4501.148	151.242	4419.175	21.948
	ReLU	3082.432	265.994	982.917	669.115	70.469	100.242
	Sigmoid	467.158	33.911	130.838	91.048	51.416	43.718
	Tanh	254.662	26.321	8.899	0.062	6.101	0.225
steps-large	Liniowa	519.519	0.572	521.099	2.648	526.716	9.486
	ReLU	437.628	26.744	286.855	168.375	165.690	251.418
	Sigmoid	429.275	13.492	113.700	19.486	49.253	3.023
	Tanh	163.526	7.194	24.658	1.777	15.391	4.435
rings5-regular	Liniowa	0.284	0.004	0.299	0.030	0.265	0.029
	ReLU	0.661	0.165	0.869	0.046	0.425	0.225
	Sigmoid	0.917	0.005	0.954	0.015	0.962	0.011
	Tanh	0.925	0.009	0.951	0.016	0.922	0.023
rings3-regular	Liniowa	0.429	0.035	0.403	0.059	0.372	0.050
	ReLU	0.847	0.066	0.960	0.007	0.836	0.146
	Sigmoid	0.953	0.006	0.977	0.004	0.969	0.014
	Tanh	0.949	0.006	0.956	0.033	0.956	0.005

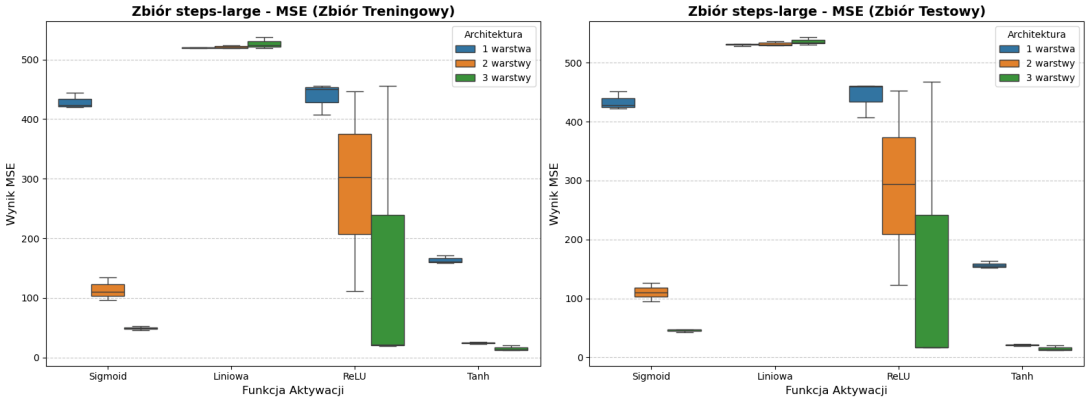
Tabela 10: Średnie finalne wartości metryk (MSE dla zbioru multimodal-large i steps-large oraz F1-Score dla pozostałych) oraz odchylenia standardowe dla konkretnych modeli dla zbiorów treningowych.

Zbiór danych	Aktywacja	1 warstwa		2 warstwy		3 warstwy	
		Średnia	Odch. std.	Średnia	Odch. std.	Średnia	Odch. std.
multimodal-large	Liniowa	4455.182	14.854	4540.651	158.181	4455.267	22.114
	ReLU	3125.401	284.010	1007.498	699.269	68.903	102.099
	Sigmoid	462.165	36.078	130.470	98.474	49.893	47.008
	Tanh	265.199	25.333	4.961	0.774	2.302	0.448
steps-large	Liniowa	530.697	1.556	532.312	4.040	536.254	6.508
	ReLU	442.771	30.516	289.882	164.585	166.880	260.513
	Sigmoid	433.577	15.367	110.586	15.720	45.612	2.570
	Tanh	156.677	5.922	21.121	1.392	14.960	4.209
rings5-regular	Liniowa	0.291	0.008	0.295	0.030	0.257	0.028
	ReLU	0.590	0.150	0.824	0.065	0.371	0.197
	Sigmoid	0.879	0.021	0.916	0.011	0.929	0.029
	Tanh	0.902	0.016	0.911	0.018	0.877	0.015
rings3-regular	Liniowa	0.415	0.042	0.397	0.069	0.359	0.048
	ReLU	0.832	0.061	0.937	0.004	0.824	0.114
	Sigmoid	0.943	0.004	0.957	0.005	0.945	0.012
	Tanh	0.937	0.007	0.935	0.030	0.938	0.004

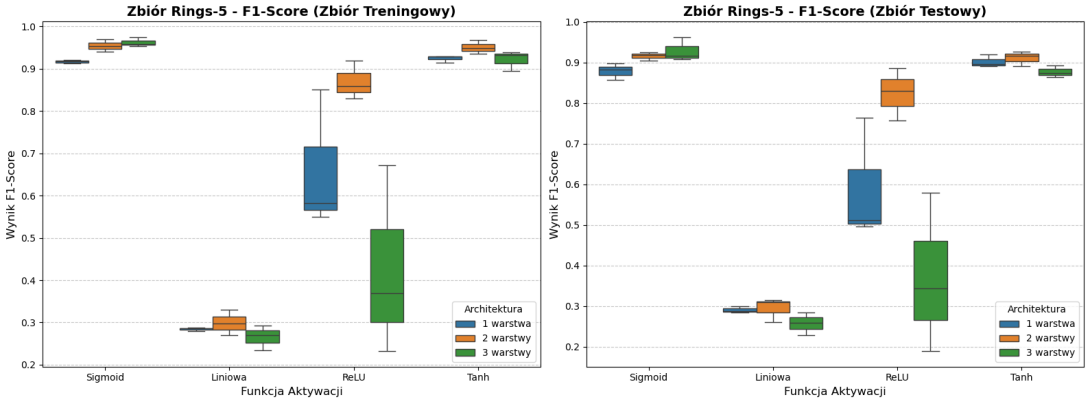
Tabela 11: Średnie finalne wartości metryk (MSE dla zbioru multimodal-large i steps-large oraz F1-Score dla pozostałych) oraz odchylenia standardowe dla konkretnych modeli dla zbiorów testowych.



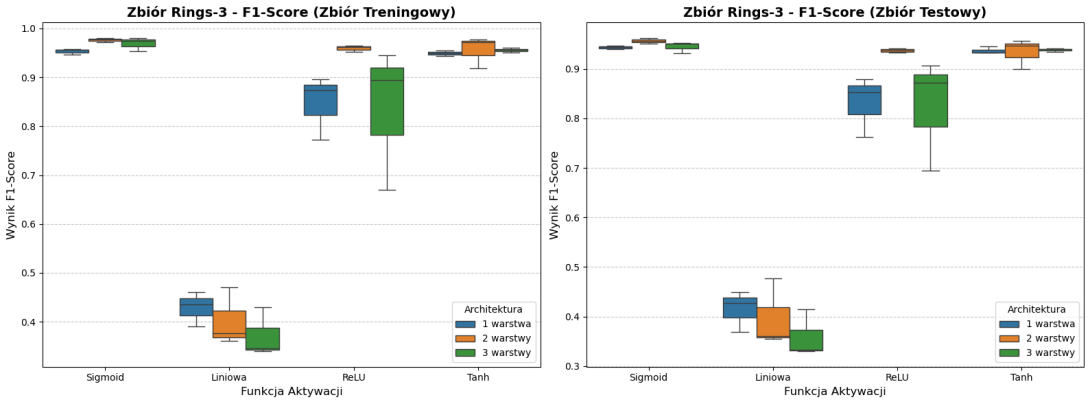
Rysunek 35: Rozkład MSE treningowego i testowego dla użytych funkcji aktywacji i architektur modelu dla zbioru multimodal-large.



Rysunek 36: Rozkład MSE treningowego i testowego dla użytych funkcji aktywacji i architektur modelu dla zbioru steps-large.



Rysunek 37: Rozkład F1-Score treningowego i testowego dla użytych funkcji aktywacji i architektur modelu dla zbioru rings5-regular.



Rysunek 38: Rozkład F1-Score treningowego i testowego dla użytych funkcji aktywacji i architektur modelu dla zbioru rings3-regular.

Zjawisko przeuczenia i regularyzacja

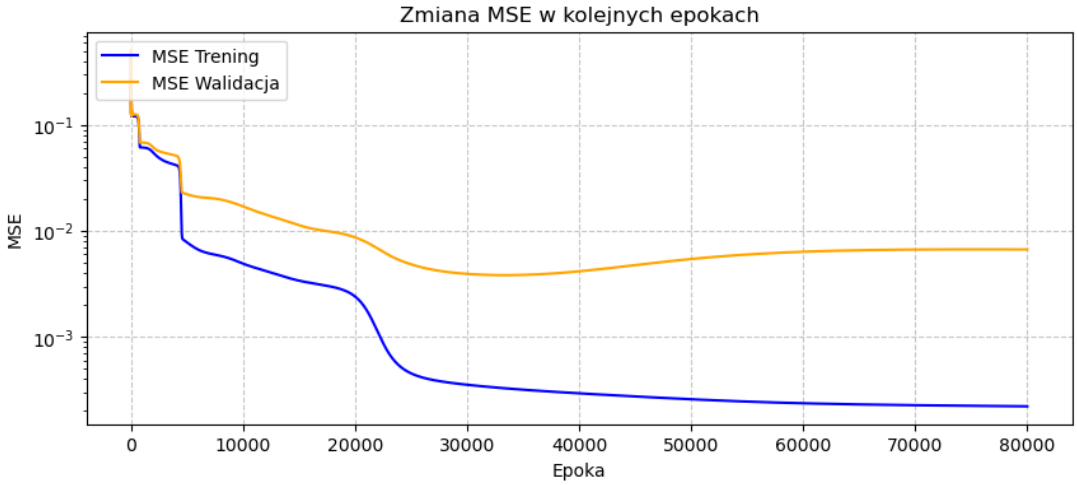
Ostatnim etapem pracy było wprowadzenie mechanizmów zapobiegających zjawisku przeuczenia. Występuje ono, gdy sieć nadmiernie optymalizuje swoje wagi pod kątem zbioru treningowego i traci zdolność generalizacji, co na wykresach funkcji straty objawia się rosnącym błędem walidacyjnym mimo ciągłego spadku błędu uczącego. Aby temu przeciwdziałać, zaimplementowany został mechanizm regularyzacji wag metodą L2, która karze model za zbyt duże wartości wag, wymuszając tworzenie łagodniejszych i bardziej uniwersalnych granic decyzyjnych. Drugim mechanizmem jest algorytm Early Stopping, który monitoruje błąd na zbiorze walidacyjnym i przerywa naukę, gdy przez wybraną liczbę epok (określaną przez parametr patience) nie następuje żadna poprawa. Zapobiega to niepotrzebnym iteracjom, a proces kończy się załadowaniem wag z optymalnego punktu uczenia.

Zbiory używane na tym etapie to multimodal-sparse, rings5-sparse, rings3-balance i xor3-balance. Aby zachować przejrzystość wyników, parametry wspólne dla wszystkich przeprowadzonych eksperymentów nie zostały ujęte w tabeli. We wszystkich badanych modelach wagi inicjowano metodą Xaviera. Proces uczenia optymalizowano za pomocą metody momentu (ze współczynnikiem pędu 0.9) oraz stałą szybkością uczenia rzędu 0.1. Uczenie odbywało się w trybie mini-batch z porcją danych wynoszącą 64 próbki. Dodatkowo, we wszystkich warstwach ukrytych zastosowano funkcję aktywacji Tanh, natomiast na warstwie wyjściowej użyto funkcji liniowej dla zadań regresji oraz funkcji Softmax dla klasyfikacji. Parametry, które ulegały zmianie w zależności od problemu i zbioru danych, zestawiono w tabeli 12.

Zbiór danych	Typ zadania	Architektura	Maks. liczba epok	Cierpliwość (patience)
multimodal-sparse	Regresja	1-64-64-1	80 000	2 000
rings5-sparse	Klasyfikacja	2-10-5-N	10 000	400
rings3-balance	Klasyfikacja	2-10-5-N	10 000	400
xor3-balance	Klasyfikacja	2-10-5-N	10 000	300

Tabela 12: Architektury modeli oraz parametry uczenia zależne od zbioru danych, gdzie N oznacza liczbę klas zbioru.

Aby zidentyfikować przeuczenie modelu, oprócz zbioru treningowego i testowego tworzony był zbiór walidacyjny, który był brany jako 20% zbioru testowego. Analizując przebieg uczenia modelu pozbawionego mechanizmów zabezpieczających, można wyraźnie zidentyfikować zjawisko przeuczenia. Na wykresie 39 pokazującym zmianę MSE w kolejnych epokach dla zbioru multimodal-sparse zauważalne jest, że chociaż błąd na zbiorze treningowym stale maleje, wartość MSE na zbiorze walidacyjnym od pewnego momentu zaczyna rosnąć. Ostateczne wartości błędów na zbiorze treningowym i testowym drastycznie się od siebie różniły, co oznacza, że sieć zbytnio dopasowała się do specyfiki danych uczących i straciła zdolność poprawnego wnioskowania dla nowych, niewidzianych wcześniej próbek. Podobne zależności można było zauważyć w reszcie zbiorów.

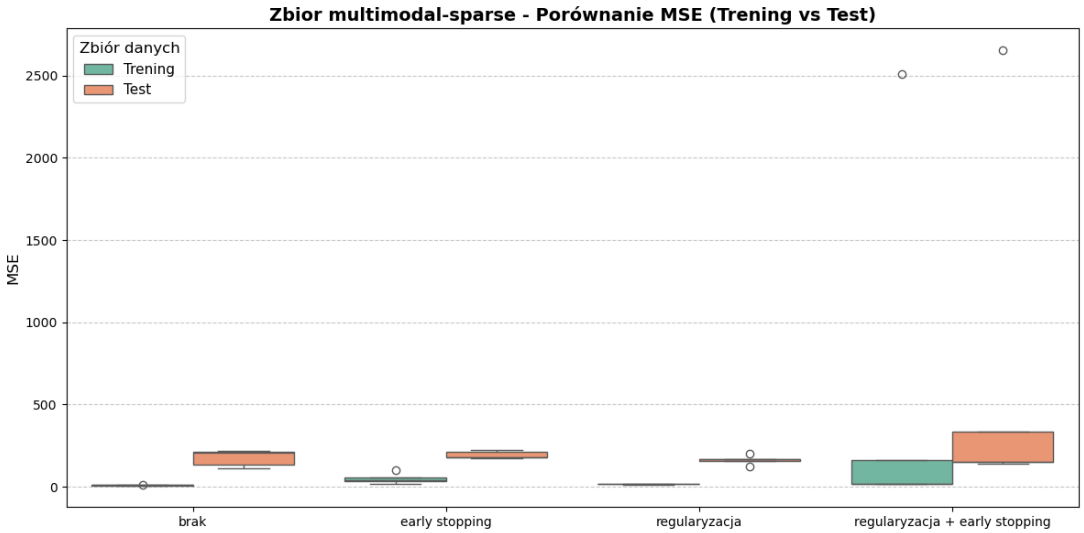


Rysunek 39: Zmiana MSE w kolejnych epokach na treningowym i walidacyjnym zbiorze multimodal-sparse

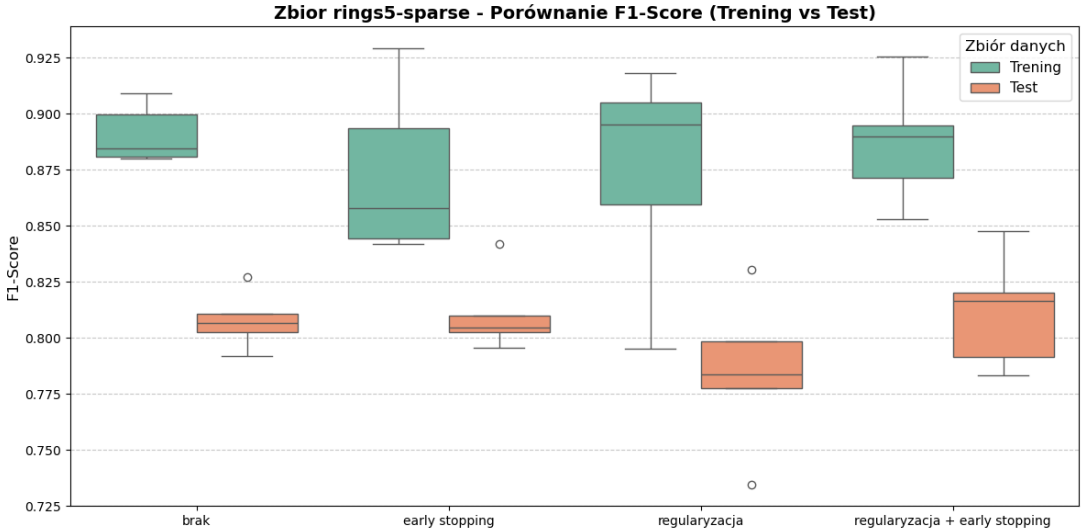
Porównane zostały 4 modele - podstawowy, używający wczesnego zatrzymania, używający regularyzacji z parametrem $l2 = 0.00001$ oraz używający zarówno wczesnego zatrzymania, jak i regularyzacji. Aby zapewnić powtarzalność wyników, zostały one wywołane dla każdego zbioru 5 razy. Otrzymane średnie wyniki metryk oraz ich odchylenia standardowe znajdują się w tabeli 13 oraz na wykresach 40, 41, 42, 43. W większości przypadków mechanizmy pozwoliły na zmniejszenie różnicy między wartością metryki na zbiorze treningowym i testowym. Największą skuteczność uzyskano, korzystając z regularyzacji, natomiast wczesne zatrzymanie pozwoliło na uzyskanie dobrych wyników w krótszym czasie. Należy zaznaczyć, że w niektórych przypadkach otrzymane wyniki nie osiągają wysoce satysfakcjonującego poziomu. Jest to bezpośredni skutek niewielkiej liczby próbek lub niezbalansowanych klas w zbiorze treningowym. Tak ograniczony zestaw danych utrudnia modelowi wyodrębnienie uniwersalnych reguł, co z kolei negatywnie odbija się na jego zdolności do generalizacji. Wartości dopasowane przez modele używające regularyzacji są widoczne na wykresach 44, 45, 46 i 47.

Zbiór danych	Konfiguracja	MSE/F1 treningowe		MSE/F1 testowe	
		Średnia	Odch. std.	Średnia	Odch. std.
multimodal-sparse	brak	9.6460	1.7976	175.2800	50.3201
	early stopping	48.6734	31.4382	192.9521	22.8083
	regularyzacja	14.7969	1.2990	162.6862	28.3702
	regularyzacja + early stopping	543.8191	1099.6137	687.1315	1102.9908
rings5-sparse	brak	0.8909	0.0130	0.8078	0.0130
	early stopping	0.8734	0.0375	0.8108	0.0181
	regularyzacja	0.8747	0.0495	0.7849	0.0347
	regularyzacja + early stopping	0.8869	0.0271	0.8118	0.0255
rings3-balance	brak	0.9209	0.0276	0.8724	0.0335
	early stopping	0.9032	0.0153	0.8502	0.0051
	regularyzacja	0.9073	0.0237	0.9151	0.0371
	regularyzacja + early stopping	0.9253	0.0168	0.9025	0.0118
xor3-balance	brak	0.9843	0.0090	0.9301	0.0231
	early stopping	0.9831	0.0121	0.9098	0.0217
	regularyzacja	0.9644	0.0200	0.9268	0.0195
	regularyzacja + early stopping	0.9732	0.0138	0.9069	0.0186

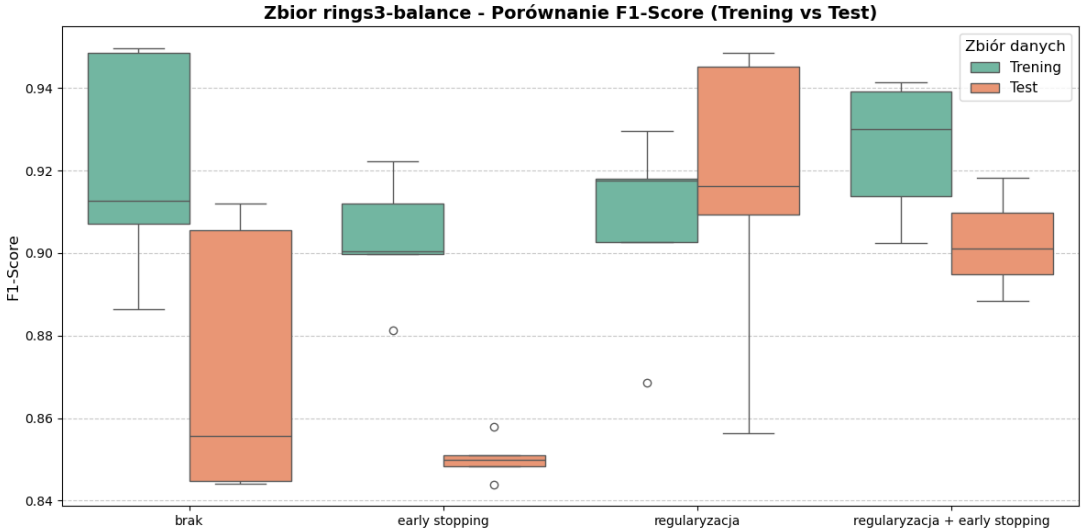
Tabela 13: Średnie finalne wartości metryk oraz odchylenia standardowe dla konkretnych modeli i zbiorów danych. Dla zbioru multimodal-sparse liczona była wartość MSE, a dla pozostałych F1.



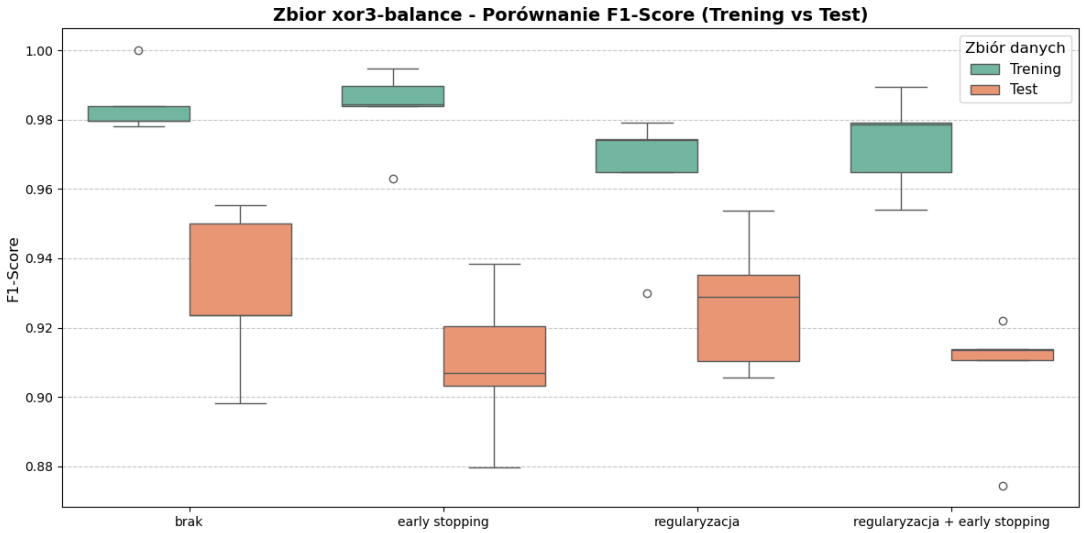
Rysunek 40: Rozkład MSE treningowego i testowego dla użytych mechanizmów zapobiegania przeuczeniu dla zbioru multimodal-sparse.



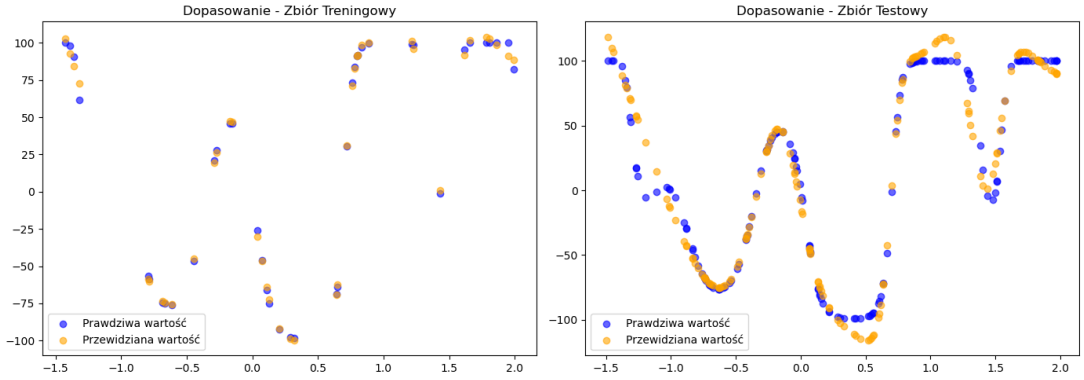
Rysunek 41: Rozkład F1-Score treningowego i testowego dla użytych mechanizmów zapobiegania przeuczeniu dla zbioru rings5-balance.



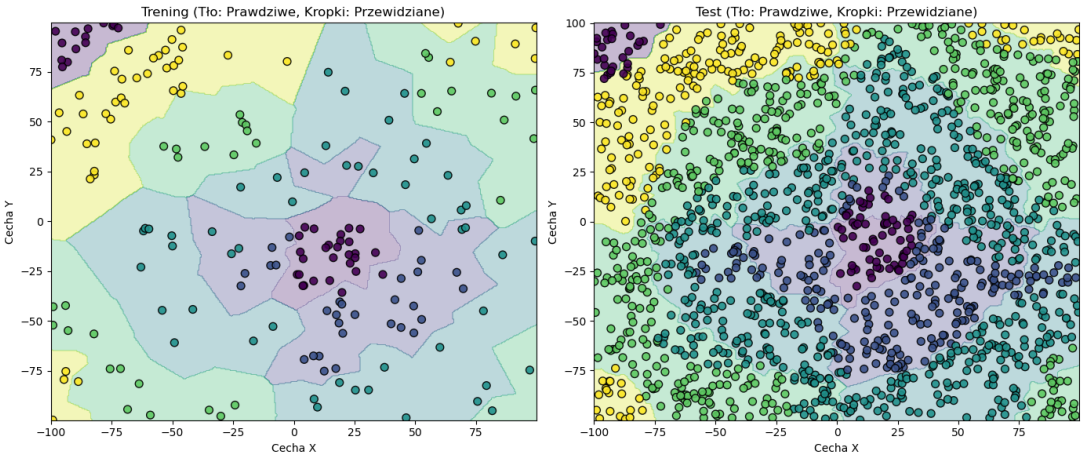
Rysunek 42: Rozkład F1-Score treningowego i testowego dla użytych mechanizmów zapobiegania przeuczeniu dla zbioru rings3-balance.



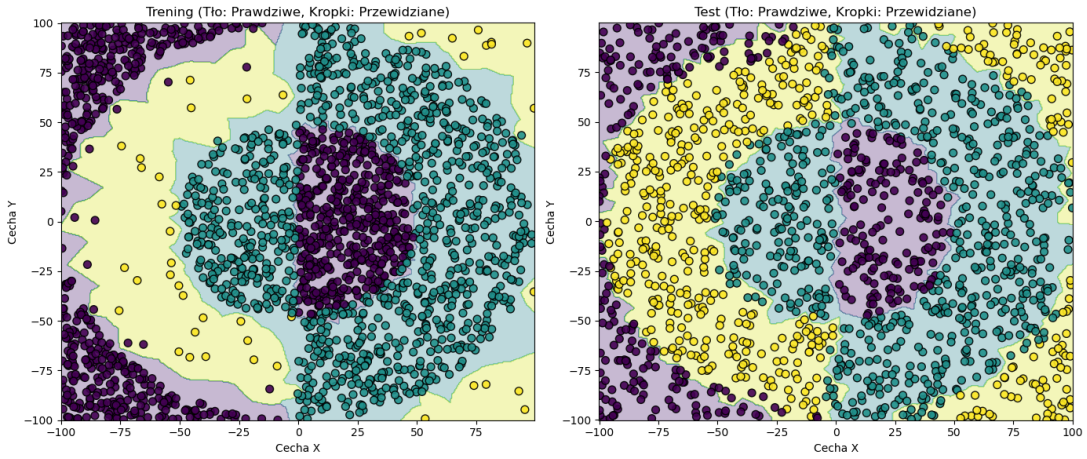
Rysunek 43: Rozkład F1-Score treningowego i testowego dla użytych mechanizmów zapobiegania przeuczeniu dla zbioru xor3-balance.



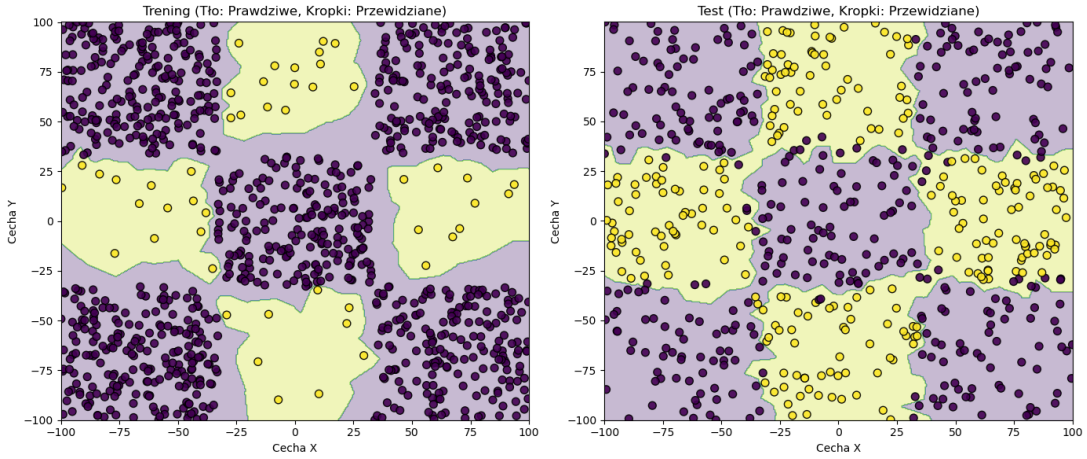
Rysunek 44: Wartości rzeczywiste i przewidziane przez model korzystający z regularyzacji dla treningowego i testowego zbioru multimodal-sparse.



Rysunek 45: Prawdziwe i przewidziane klasy przez model korzystający z regularyzacji dla treningowego i testowego zbioru rings5-sparse.



Rysunek 46: Prawdziwe i przewidziane klasy przez model korzystający z regularyzacji dla treningowego i testowego zbioru rings3-balance.



Rysunek 47: Prawdziwe i przewidziane klasy przez model korzystający z regularyzacji dla treningowego i testowego zbioru xor3-balance.

Wnioski i podsumowanie

W ramach niniejszego projektu z powodzeniem zaimplementowano od podstaw w pełni funkcjonalny model sztucznej sieci neuronowej - perceptron wielowarstwowy. Stopniowa rozbudowa bazowej architektury pozwoliła na stworzenie wszechstronnego narzędzia, zdolnego do skutecznego rozwiązywania zarówno problemów regresji, jak i wieloklasowej klasyfikacji. Przeprowadzone na wielu zbiorach danych eksperymenty dostarczyły wniosków dotyczących wpływu poszczególnych hiperparametrów i mechanizmów na proces trenowania sieci.

Opracowano różne metody inicjalizacji wag (takie jak rozkład jednostajny, metody Xaviera i He) oraz wdrożono algorytm wstecznej propagacji błędu. Testy udowodniły, że aktualizacja wag w trybie Mini-Batch jest znacznie efektywniejsza niż klasyczne podejście Full-Batch, ponieważ pozwala na szybsze uczenie i skuteczniejsze omijanie minimów lokalnych. Proces optymalizacji wag uległ znacznej poprawie po zastosowaniu zaawansowanych algorytmów. Metoda momentu gwarantowała błyskawiczną zbieżność i szybkie spadki funkcji straty, podczas gdy algorytm RMSProp charakteryzował się niezwykłą stabilnością w późniejszych fazach treningu.

Istotnym elementem analizy było również porównanie funkcji aktywacji. Wyniki wykazały bezwzględną przewagę tangensa hiperbolicznego, który najszybciej i najpewniej prowadził do optymalnych rezultatów. Z kolei funkcja Sigmoid pozwalała na poprawne wyuczenie modelu, lecz wymagała więcej czasu i głębszych architektur. Algorytm ReLU cechował się zauważalną wariancją wyników, natomiast funkcja liniowa ostatecznie potwierdziła brak zdolności do modelowania nieliniowych problemów w warstwach ukrytych. Skupiając się na zadaniach klasyfikacji, udowodniono absolutną konieczność zastosowania na warstwie wyjściowej funkcji Softmax w połączeniu z funkcją straty entropii krzyżowej.

W końcowych fazach prac wyeliminowano problem podatności rozbudowanych modeli na zjawisko przeuczenia. Zauważono, że wystarczająco duże sieci miały tendencję do zapamiętywania danych treningowych, co skutkowało utratą zdolności generalizacji i wzrostem błędu na zbiorach walidacyjnym oraz testowym. Zaimplementowanie kary za zbyt

duże wagi w postaci regularyzacji L2 oraz mechanizmu wczesnego zatrzymywania skutecznie zredukowało ten problem, wymuszając na modelach poszukiwanie bardziej uniwersalnych i łagodnych granic decyzyjnych.

Wszystkie te stworzone moduły połączyły się w spójną, stabilną i konfigurowalną sieć neuronową.

Zakres użycia LLM

W trakcie realizacji projektu oraz przygotowywania sprawozdania wykorzystano model językowy Gemini. Służył on jako wsparcie w procesie korekty stylistycznej i gramatycznej tekstu. Dodatkowo model wykorzystano do konsultacji programistycznych przy optymalizacji i debugowaniu tworzonego od podstaw kodu.